[BUSINESS] FORECASTING AND PREDICTIVE MODELING

LECTURE 8

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

In Lecture 7 ...

- The forecasting workflow
  - Simple (baseline) forecasting models
  - Fitted values and residuals
  - Prediction intervals
  - Forecasting with decomposition
  - Evaluating the performance of the forecasting model

Today

- The forecasting workflow
  - Time series cross-validation
- Time series regression models

Recap I

- Simple forecasting models:
  - Mean model - mean of historical observations
  - Naive model - repeat the last observation
  - Seasonal naive model - repeat the last season
  - Drift model - draw a line between the first and last observation and continue this line for future

Recap II

- Fitted values and residuals
  - Once you have built a model using some historical data, apply that model on the historical data (do 1 step forecasts for every possible period) $\rightarrow$ fitted values
  - The difference between the actual value and the fitted value is the residual.

Recap III

- Prediction intervals
  - Forecasts always come with a prediction interval, which tells us how certain we are in the forecast. The point forecast is in general the middle point of the interval.
  - In general, the larger the forecast horizon, the wider the prediction interval.
  - Also, if we want higher certainty (for example 90% prediction interval instead of 80%) the prediction interval will be wider.

Recap IV

- Performance measures
  - Scale dependent performance measures: MAE, RMSE
  - Scale independent performance measures: MAPE, sMAPE, MASE
- For proper evaluation of a forecasting model, it should be evaluated on data which was not used for building the model. One way to do this is to split the data in train test sets (for example, forst 80% of observations are used for training, the last 20% for testing).

Time series cross-validation I

- Instead of doing a simple train-test split, in general, cross-validation is considered a better evaluation method.
- In case of cross validation, we have several training sets of increasing size and the test set is always the next observation (also called *evaluation on a rolling forecasting origin*). For example, for a time series with 20 observations, we could have the following cross validation-scenario:
  - Use the first 5 observations for training and observation 6 for testing (the rest is unused)
  - Use the first 6 observations for training and observation 7 for testing
  - Use the first 7 observations for training and observation 8 for testing
  - ...
  - Use the first 19 observations for training and observation 20 for testing

Time series cross-validation II

- For each test case we will have an error (or accuracy) measure and their average is the performance of the method.
- Cross-validation can be performed for multi-step forecasts as well. For example, you can use the first X observations for training and forecast instance $X + 4$ (but only one instance).
- **Obs.** It needs to be carefully analyzed how many observations are needed initially (on one hand you would like to do as many rounds as possible, on the other hand, you cannot train a good model on too few training instances).

Time series cross-validation III

- In R, cross-validation is implemented in the *strech_tsibble* function, which creates many training sets. You need to pass it the initial training set length, *.init* (i.e. the minimum number of observations to use for training) and the increase in training set size, *.step*.

```
google_2015_tr <- googleTrain |>
  stretch_tsibble(.init = 3, .step = 1)
google_2015_tr |> view()

google_2015_tr |>
  model(RW(Close ~drift())) |>
  forecast(h = 1) |>
  accuracy(googleTrain)
```

```
  .model               .type    ME  RMSE   MAE   MPE  MAPE  MASE RMSSE   ACF1
  <chr>                <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
1 RW(Close ~ drift()) Test  0.726  11.3  7.26 0.112  1.19  1.02  1.01 0.0985
```

Time series regression models

- The main idea of time series regression models is to forecast the value of a time series y assuming that it has a linear relationship with another time series x.
- We will then use the value of x (called the *predictor variable*) to forecast the value of y (the *forecast variable*).

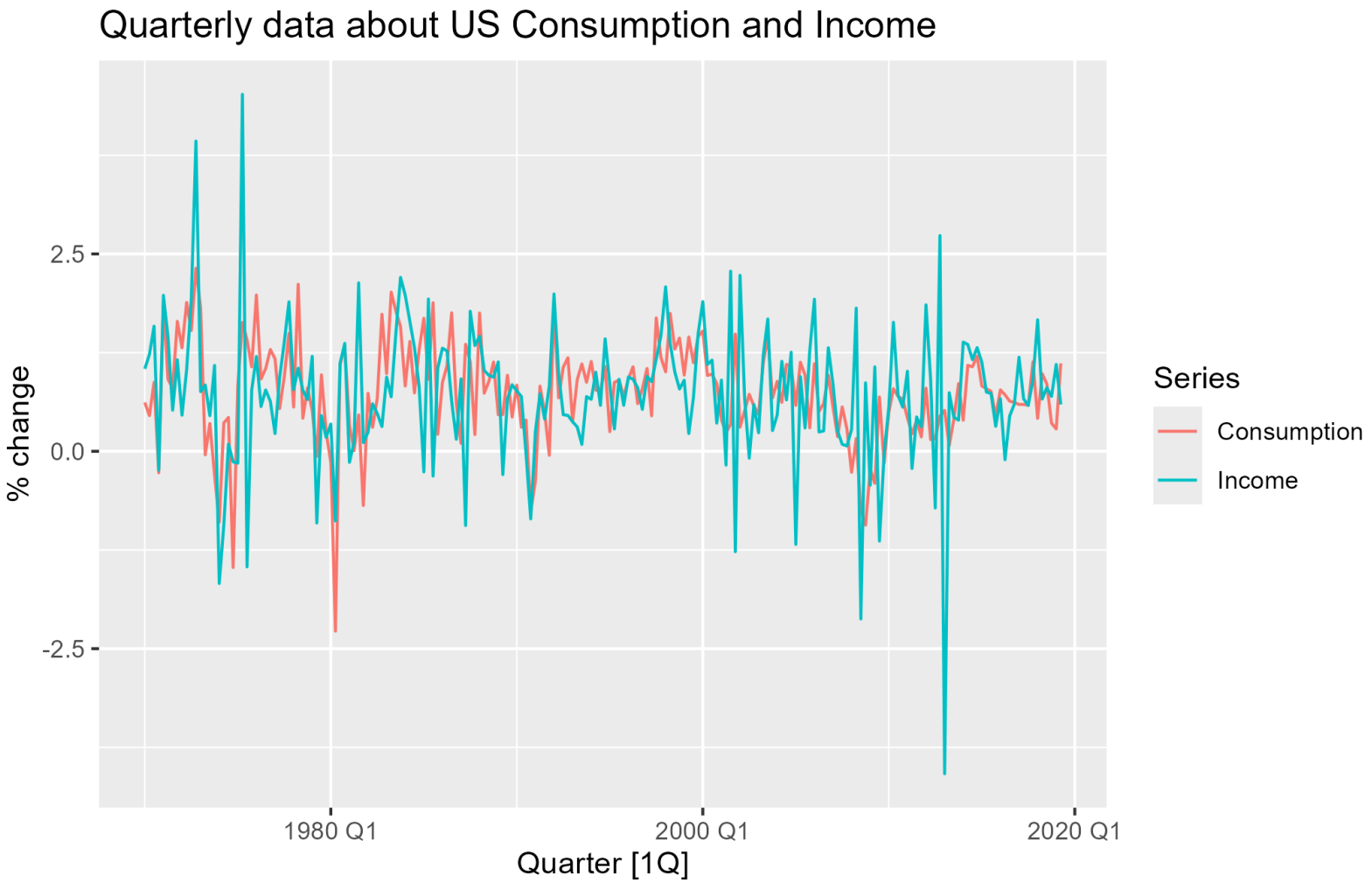
Simple linear regression

- In the simplest case, we have a linear relationship between the forecast variable and one single predictor variable:

$$y_t = \beta_0 + \beta_1 * x_t + \epsilon_t$$

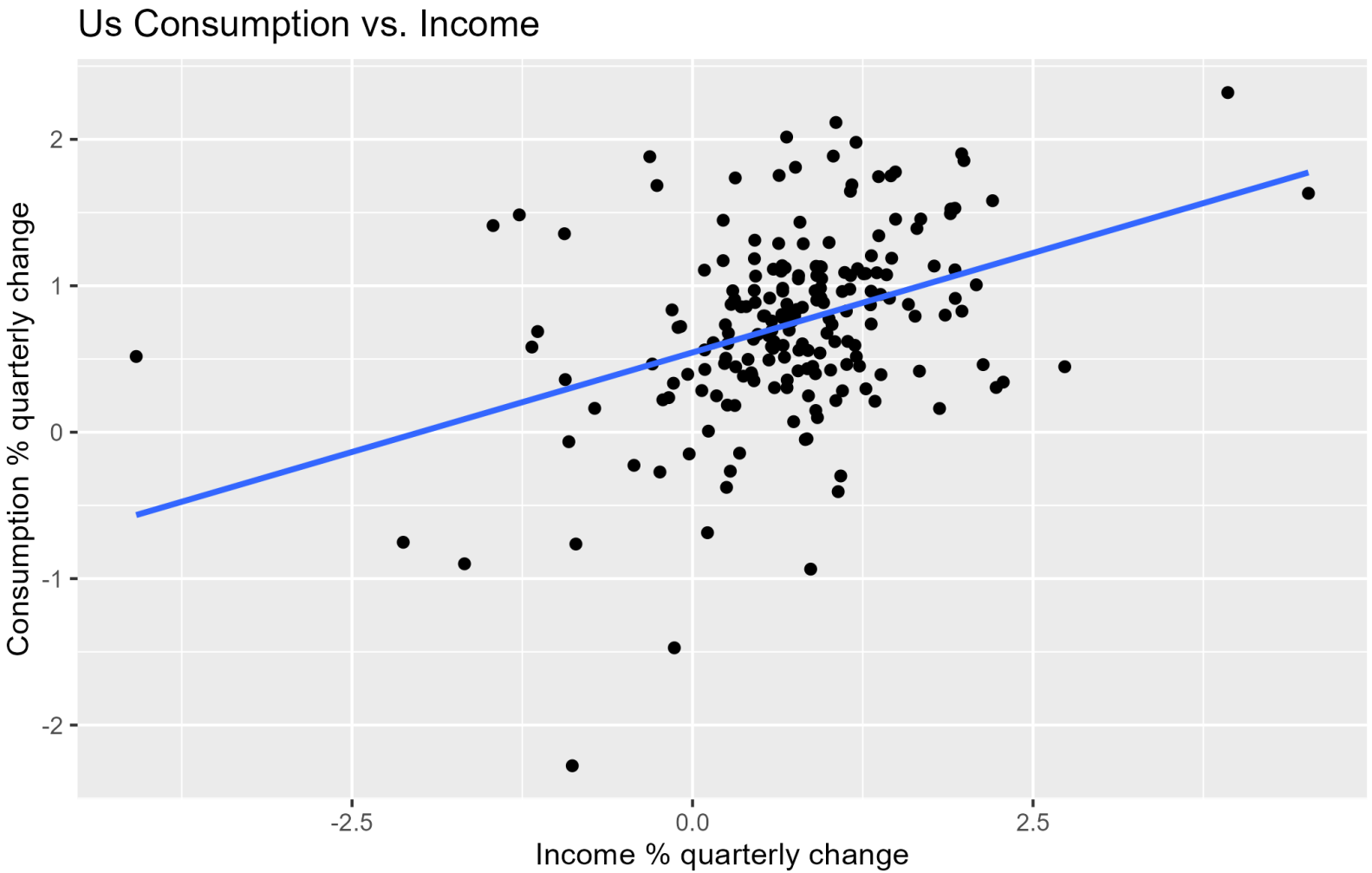
- where $\beta_0 + \beta_1 * x_t$ is called the *explained* part of the model and ϵ_t is the *random error* part.
- Let's consider the *us_change* data set, containing quarterly information about the percentage change in income, production, savings, consumption and unemployment from 1970 to 2019.
- Let's plot the Consumption and the Income from it.

```
us_change |>
 pivot_longer(c(Consumption, Income), names_to = "Series") |>
 autoplot(value) +
 labs(title = "Quarterly data about US consumption and income",
   y = "% change")
```


Quarterly data about US Consumption and Income
% change
2.5
0.0
-2.5
1980 Q1
2000 Q1
2020 Q1
Quarter [1Q]
Series
Consumption
Income

- We can see that the two time series often move together in the same direction. Let's do a scatter plot of consumption against income.

```
us_change |>
  ggplot(mapping = aes(x = Income, y = Consumption)) +
 labs(title = "Us consumption vs. income",
       x = "Income % quarterly change",
       y = "Comsumption % quarterly change") +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```


Us Consumption vs. Income
Consumption % quarterly change
2
1
0
-1
-2
-2.5
0.0
2.5
Income % quarterly change

- In order to see the exact equation we can use the *TSLM* function, passed to the model function.

```r
usChange_lm <- us_change |>
  model(TSLM(Consumption ~ Income)) |> #notation means: build model to predict
     Consumption using Income as predictor

usChange_lm |> tidy()

usChange_lm |>report()
```

```
Series: Consumption
Model: TSLM

Residuals:
     Min       1Q   Median       3Q      Max
-2.58236 -0.27777  0.01862  0.32330  1.42229

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.54454    0.05403  10.079  < 2e-16 ***
Income       0.27183    0.04673   5.817  2.4e-08 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.5905 on 196 degrees of freedom
Multiple R-squared: 0.1472, Adjusted R-squared: 0.1429
F-statistic: 33.84 on 1 and 196 DF, p-value: 2.4022e-08
```

- From the output (section Coefficients), we can see that the linear regression model is

$$\hat{y}_t = 0.54 + 0.27 * x_t$$

Multiple linear regression

- Sometimes we might have more than one predictor variable(s), in these cases we have multiple linear regression:

$$y_t = \beta_0 + \beta_1 * x_{1,t} + \beta_2 * x_{2,t} + \cdots + \beta_k * x_{k,t} + \epsilon_t$$

- The coefficient β_i measures the effect of predictor x_i after taking into account the effects of all the other predictors.
- When using a linear regression model, we automatically make some assumptions about the above variables:
  - there is a linear relation between the predictor(s) and the forecast variable
  - the errors have zero mean and they have no autocorrelation and are not related to the predictor variables

- Our *us_production* data set contains other time series as well, which might be relevant for forecasting consumption: personal savings and unemployment rate.
- We can use them in a model as predictors:

```
us_change |>
  model(TSLM(Consumption ~ Income + Savings)) |> report()
```

```
Series: Consumption
Model: TSLM

Residuals:
     Min       1Q   Median       3Q      Max
-0.96934 -0.14829 -0.01167  0.14240  1.41274

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.21543    0.03393   6.349 1.49e-09 ***
Income       0.83664    0.03748  22.324  < 2e-16 ***
Savings     -0.05901    0.00282 -20.927  < 2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.3286 on 195 degrees of freedom
Multiple R-squared: 0.7373, Adjusted R-squared: 0.7346
F-statistic: 273.6 on 2 and 195 DF, p-value: < 2.22e-16
```

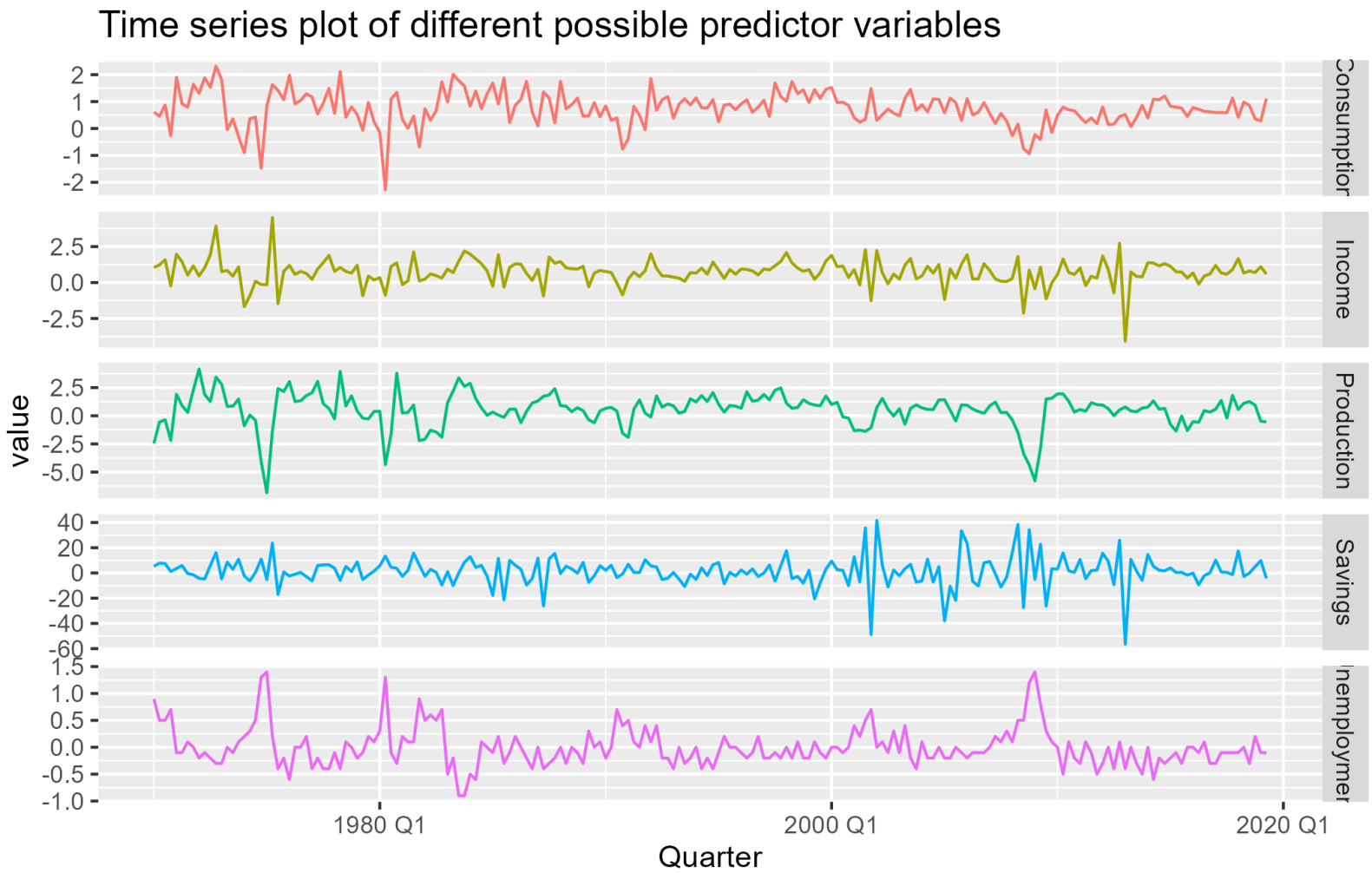
- The report function tells us that the regression model is the line:

$$\hat{y}_t = 0.22 + 0.84 * Income_t - 0.06 * Savings_t$$

- When we have several possible predictors, it might be a good idea to first do some visualizations, to get an intuition about which ones to add to the model.
- We can do simple time series plots in parallel for them (to visually evaluate correlation) or we can do scatterplots.

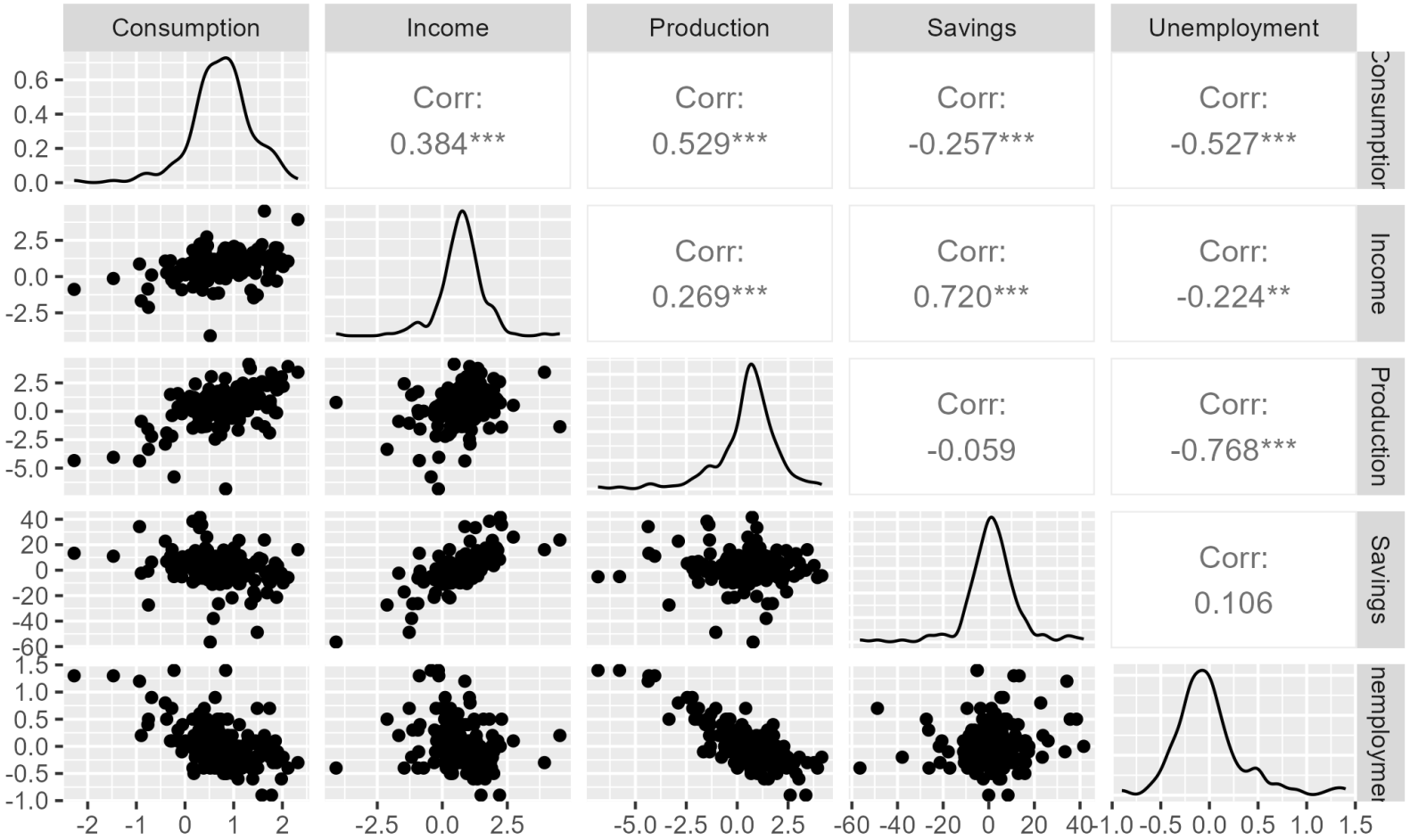
```
us_change |>
  pivot_longer(-Quarter, names_to = "Name") -> us_change_long

us_change_long |>
  ggplot(mapping = aes(x = Quarter, y = value, color = Name)) +
  geom_line() +
  facet_grid(vars(Name), scales = "free_y") +
  guides (colour = "none") +
  labs(title = "Time series plots of different possible predictor variables")
```


Time series plot of different possible predictor variables
value
Consumption
Income
Production
Savings
Unemployment
2
1
0
-1
-2
2.5
0.0
-2.5
2.5
0.0
-2.5
-5.0
40
20
0
-20
-40
-60
1.5
1.0
0.5
0.0
-0.5
-1.0
1980 Q1
2000 Q1
2020 Q1
Quarter

```
us_change |>
  GGally::ggpairs(columns = 2:6) +
  labs(title = "Scatterplots and correlations for pairs of different possible
      predictor variables")
```

Scatterplots and correlations for pairs of different possible predictor variables



- We have a few high correlations. Should we trust them?

Least squares estimation

- For finding the values of the coefficients $\beta_1, \beta_2, etc.$ a method called *least squares estimation* is used. This method tries to reduce the sum of the squares of the error term:

$$\sum_{i=1}^{T} \epsilon_t^2 = \sum_{i=1}^{T} (y_t - \beta_0 - \beta_1 * x_{1,t} - \beta_2 * x_{2,t} - \cdots - \beta_k * x_{k,t})^2$$

- Determining the best values for the coefficients is called *fitting* or *training* the model.
- In R the *TSLM* function can be used to fit a linear regression model.

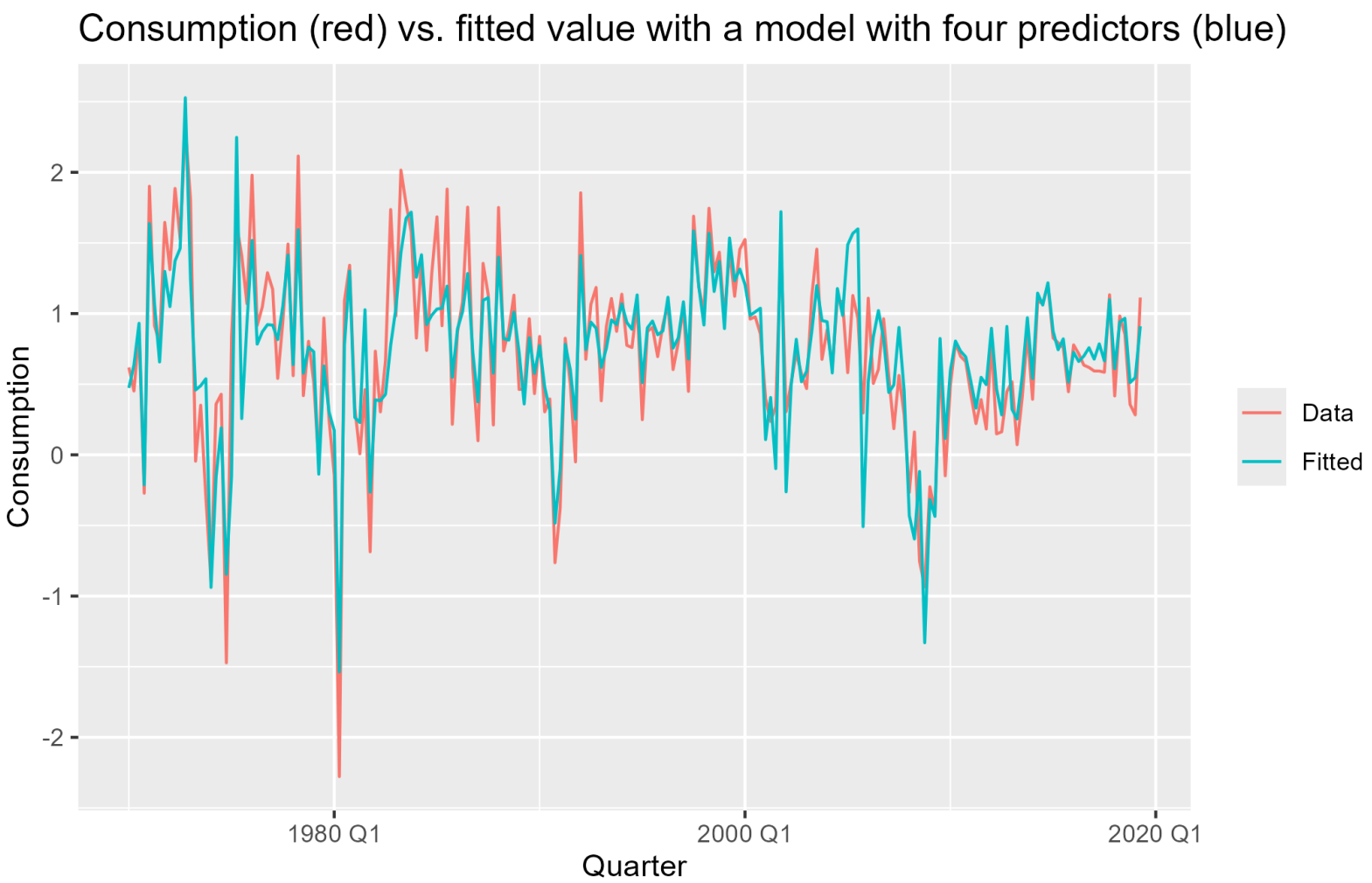
Fitted values I

- The fitted values (predictions over the training data) can be computed using the coefficients provided by the model and setting the error term to zero.

- Remember, the *augment* function will display the fitted values and the residuals (which is the error term of the equation)

```
us_change |>
  model(TSLM(Consumption ~ Income + Savings + Unemployment + Production)) |>
  augment() |>
  ggplot(mapping = aes(x = Quarter)) +
  geom_line(mapping = aes(y = Consumption, color = "Data")) +
  geom_line(mapping = aes(y = .fitted, color = "Fitted")) +
  guides(color = guide_legend(title = NULL)) +
  labs(title = "Consumption (red) vs. fitted value with a model with four
      predictors (blue)")
```

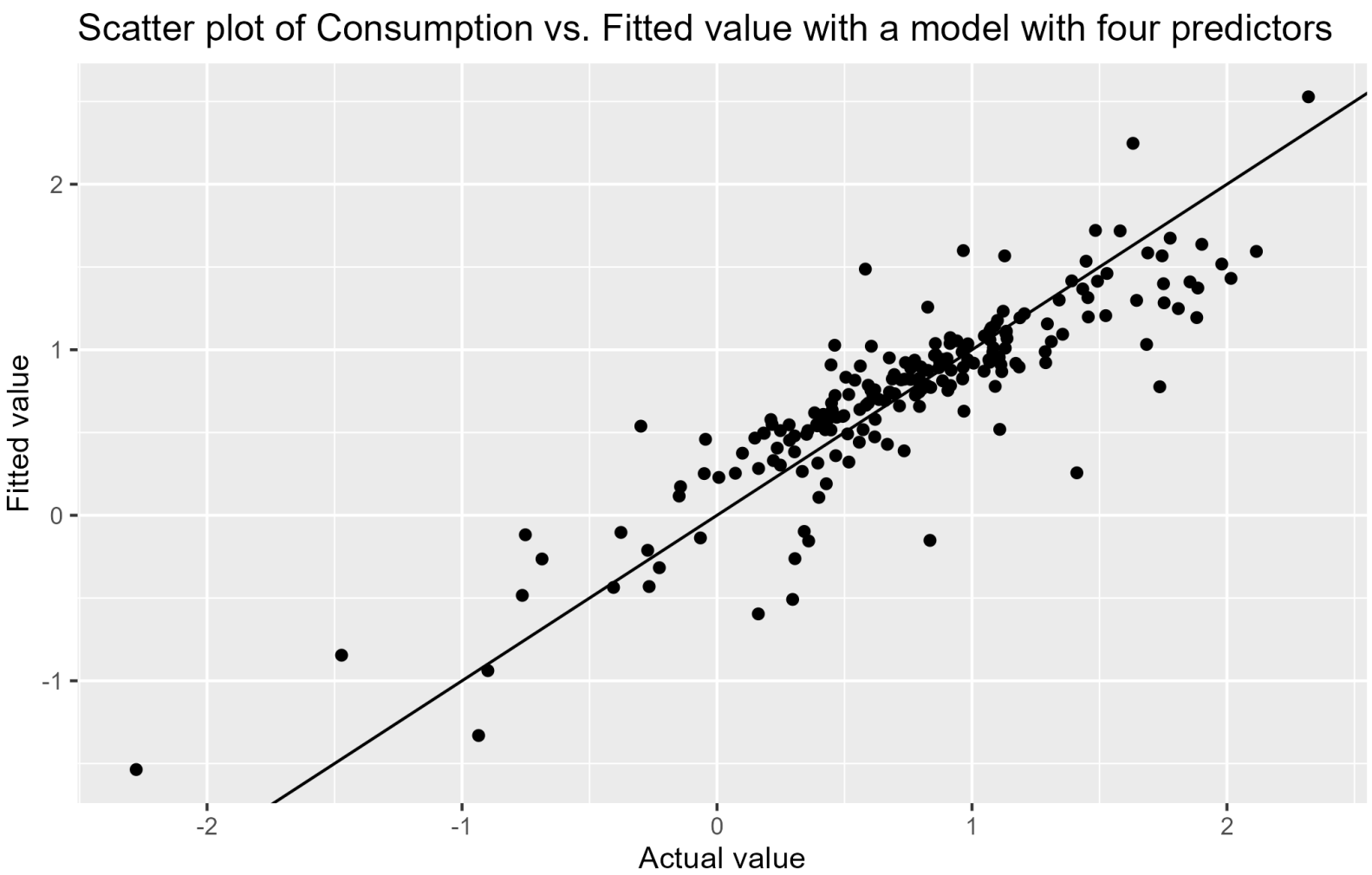
Fitted values II



Fitted values III

```
us_change |>
  model(TSLM(Consumption ~ Income + Savings + Unemployment + Production)) |>
  augment() |>
  ggplot(mapping = aes(x = Consumption, y = .fitted)) +
  geom_point() +
  labs(x = "Actual value", y = "Fitted value") +
  geom_abline(intercept = 0, slope = 1) +
  labs(title = "Scatter plot of Consumption vs. fitted value with a model with
      four predictors (Income, Savings, Unemployment, Production")
```

Fitted values IV



Goodness of fit

- A commonly used function to see how well a linear model fits the set of training data is the R^2. It can be computed in the following way:

$$R^2 = \frac{\sum(\hat{y}_t - \bar{y})^2}{\sum(y_t - \bar{y})^2}$$

- where $\bar{y}$ is the mean of the values from the training data.
- One way to interpret R^2 is to say that it measures the proportion of the variation in the forecast variable that is explained by the model.
- R^2 takes values between 0 and 1, the value 1 meaning perfect fit for the model. Since R^2 is defined on the train data, proper evaluation with test set and forecasts is needed.
- In R, the value of R^2 can be seen in the output of the *report* function.

Residual standard error

- Another measure to see how well the data fits is the *residual standard error* computed in the following way:

$$\hat{\sigma}_e = \sqrt{\frac{1}{T-k-1} * \sum_{t=1}^{T} e_t^2}$$

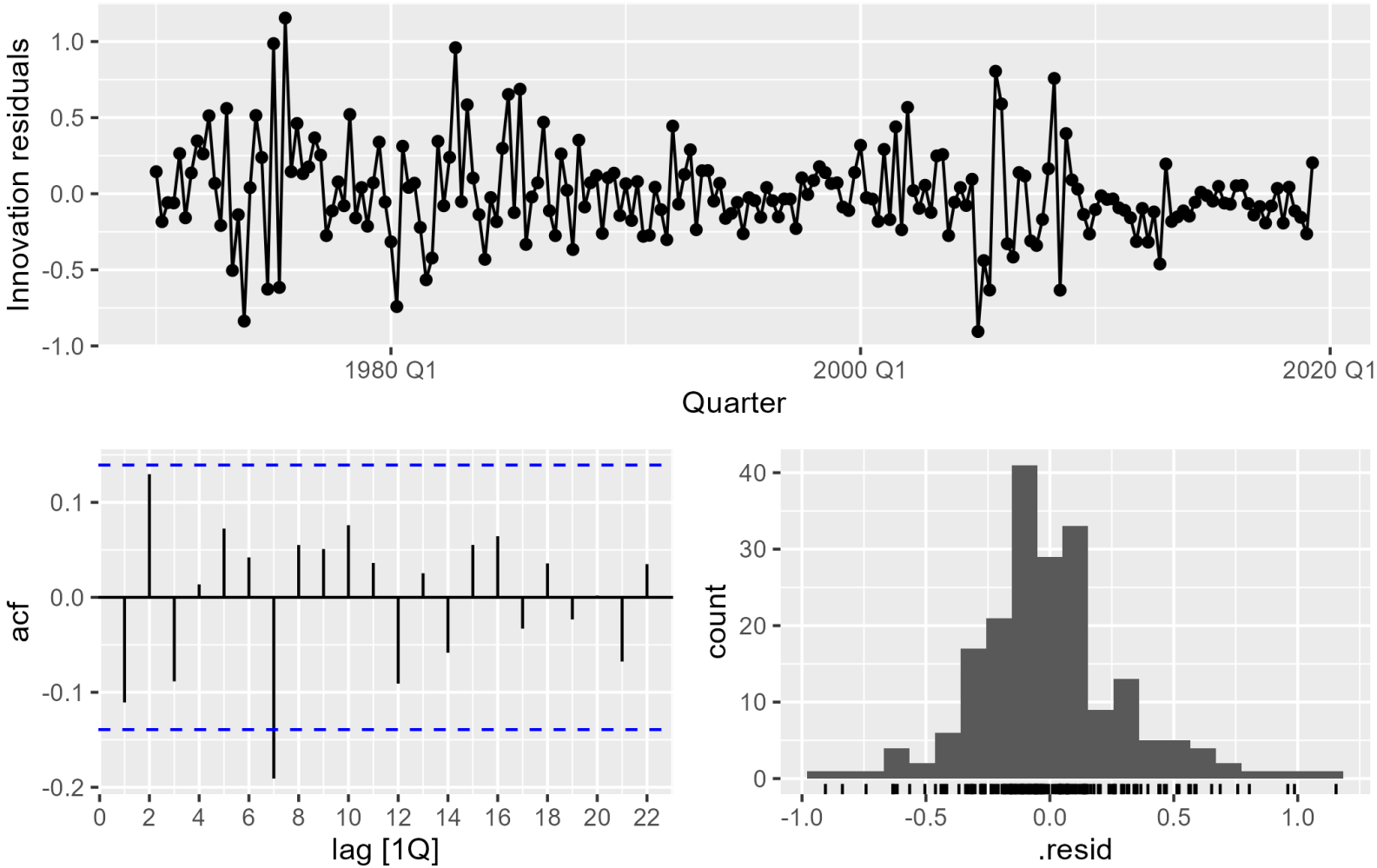
- where k is the number of predictors in the model.
- The *report* function in R will return the value of the *residual standard error* as well (besides the value of R^2).

Evaluating the regression model I

- What was discussed in the previous lesson about evaluating a model is valid here as well; we can do residual diagnostics, using the *gg_tsresiduals* function:
  - ACF plot (possibly with statistical test)
  - Histogram of the residuals
  - plot of the residuals

```
us_change |>
  model(all = TSLM(Consumption ~ Income + Savings + Unemployment + Production))
    |>
  gg_tsresiduals()
```

Evaluating the regression model II

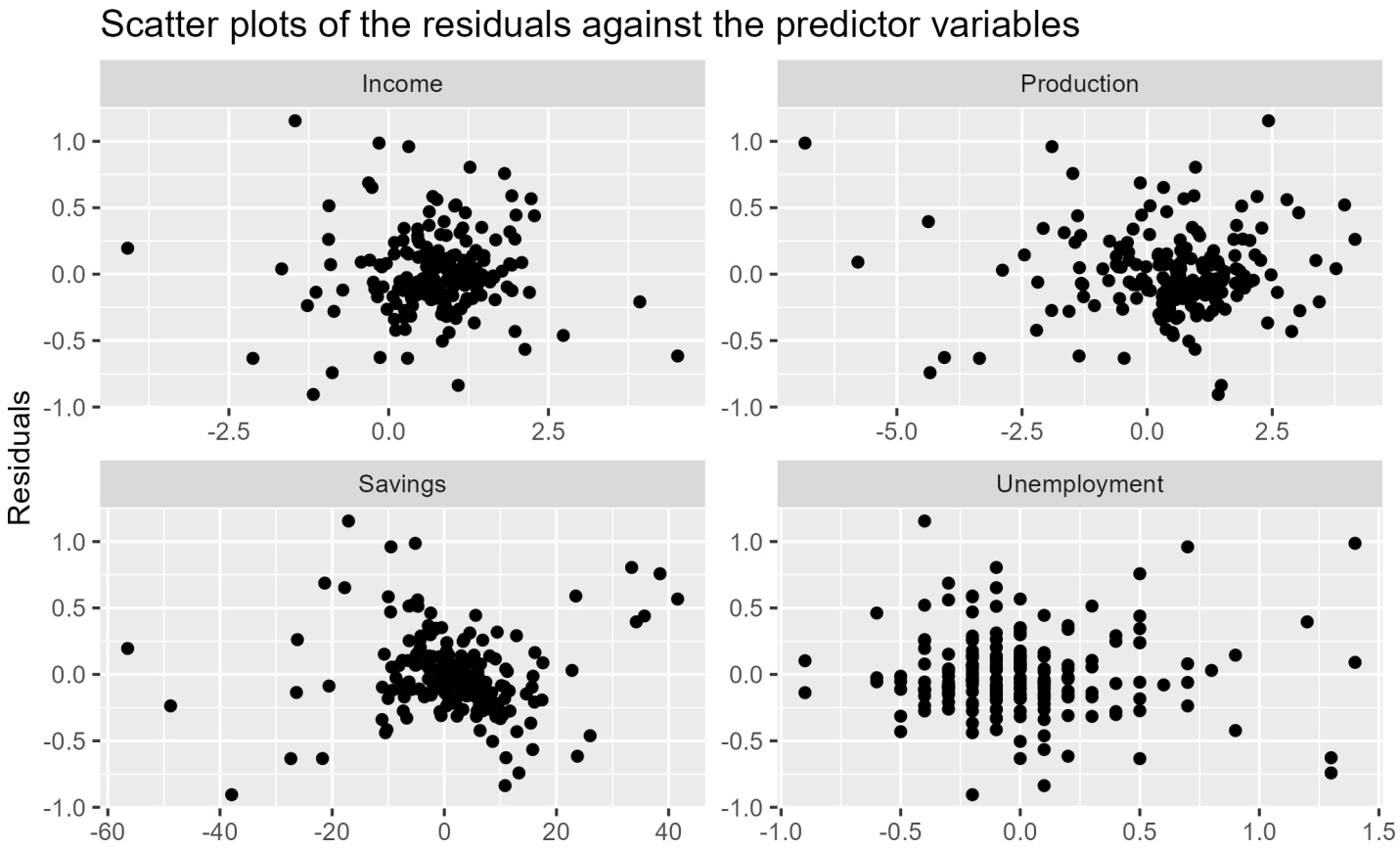


Evaluating the regression model III

- Another thing that can be done is to create scatter plot of the residuals against the predictor variables. If they show a pattern then it might happen that the predictor should be included in a non-linear form.
- Also, if there are predictors not included in the model, they should also be plot against the residuals. If there is a pattern, these predictors should be added, but possibly in a non-linear form.

```
residuals <- us_change |>
  model(all = TSLM(Consumption ~ Income + Savings + Unemployment + Production))
    |>
  augment()

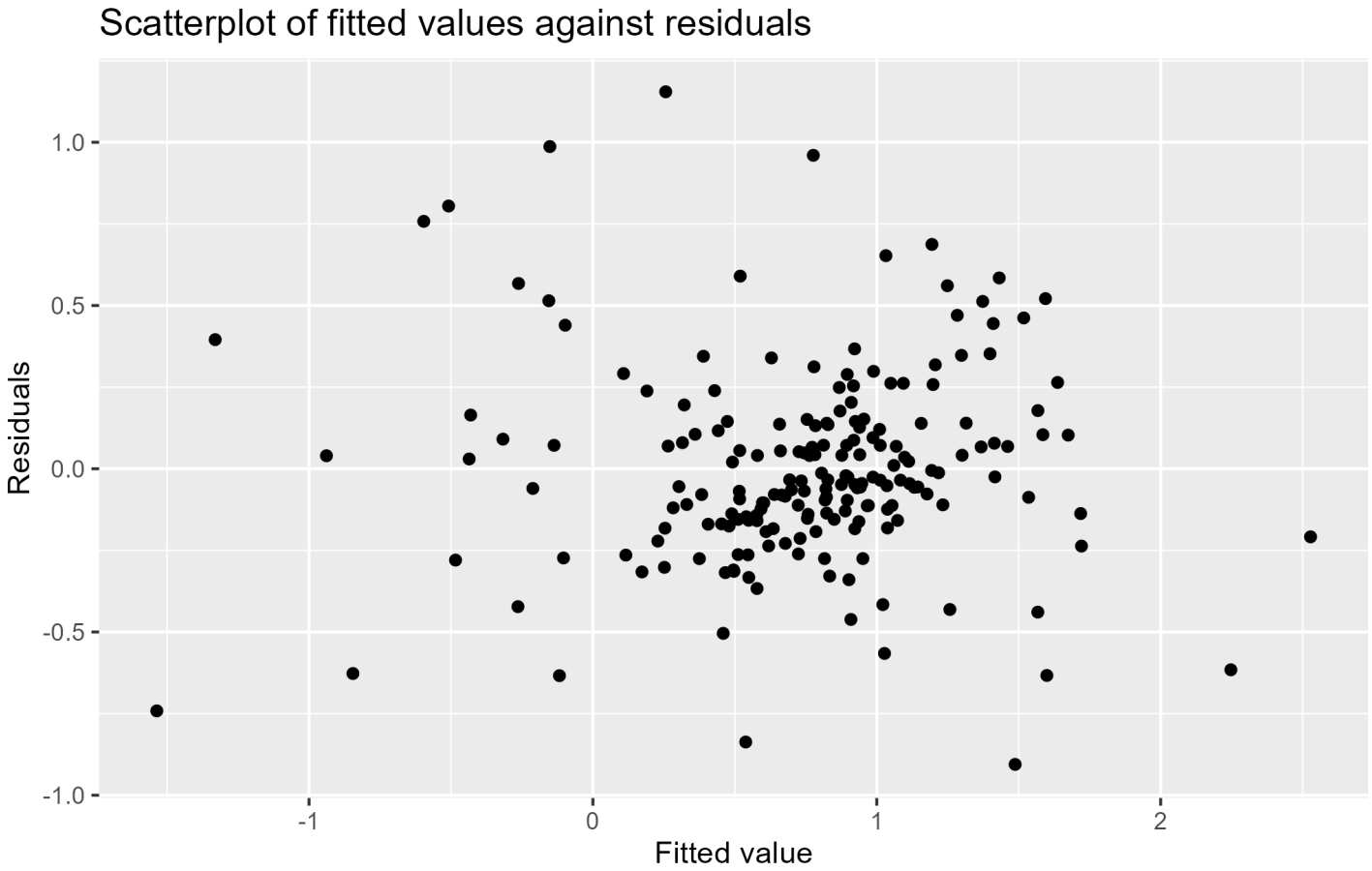
us_change |>
  left_join(residuals, by = "Quarter") |>
  pivot_longer(c(Income, Production, Savings, Unemployment), names_to = "
    predictor", values_to = "x") |>
  ggplot(mapping = aes(x = x, y = .resid)) +
  geom_point() +
  facet_wrap(vars(predictor), scales = "free") +
  labs(y = "Residuals", x = "", title = "Scatter plots of the residuals against
    the predictors of the model.")
```



- We can see that there is no pattern in either of the scatterplots.

- We can also plot the residuals against the fitted values, and there should be no pattern there either.
- If there is a pattern, maybe a transformation of the forecast variable is required.

```
residuals |>
  ggplot(mapping = aes(x = .fitted, y = .resid)) +
  geom_point() +
  labs(x = "Fitted value",
    y = "Residuals",
    title = "Scatterplot of fitted values against residuals")
```



- There is no pattern in the scatterplot.

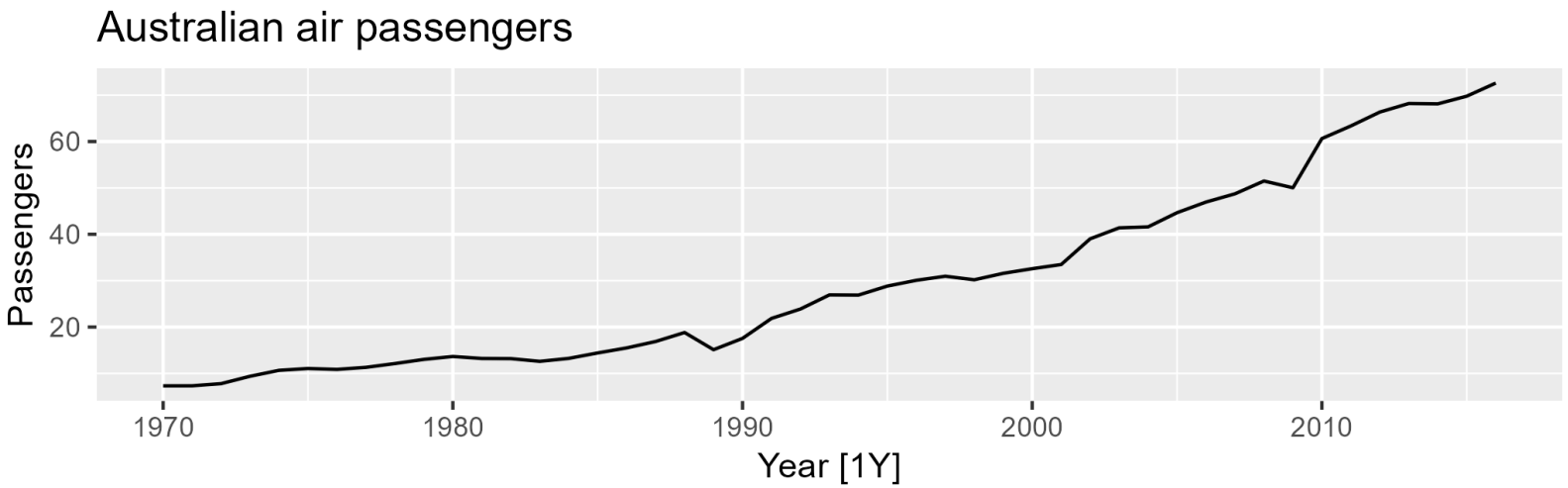
Spurious regression

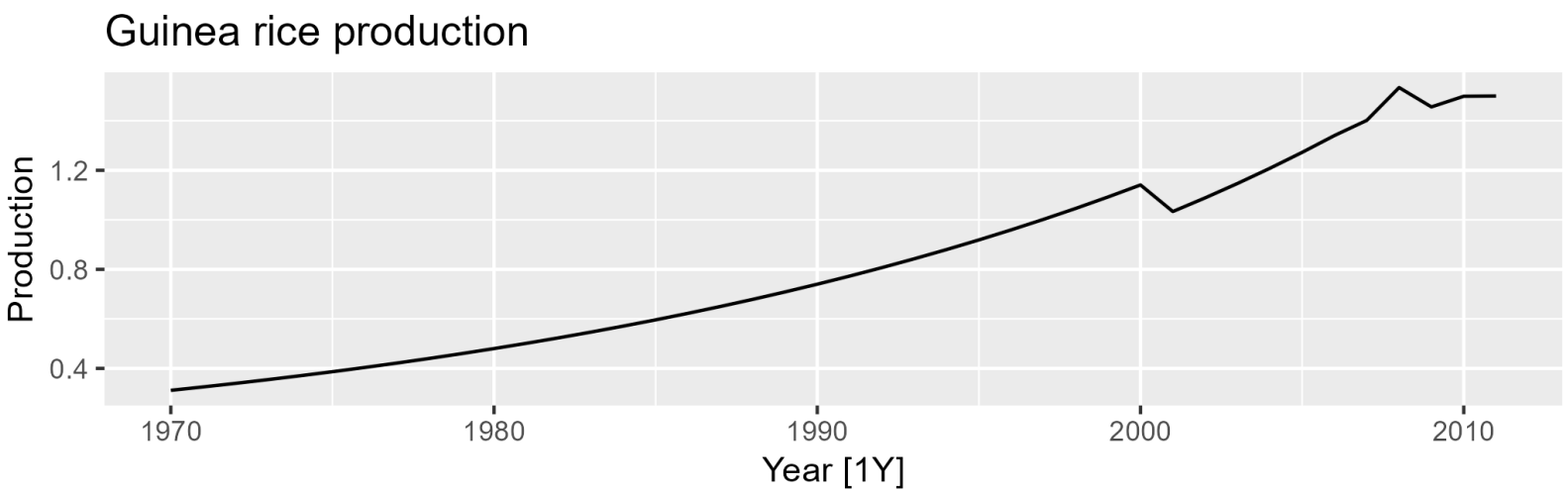
- We have talked about spurious correlation previously: the phenomenon that two time series are correlated (have a high correlation coefficient), simply because they have a similar trend.
- Let's take two time series that have nothing to do with each other: *aus_airpassengers* - the total annual air passengers in million between 1970 and 2016 - and *guinea_rice* - the total annual rice production for Guinea 1970 - 2011.
- If we plot them we will get very similar plots

```r
cor(aus_airpassengers$Passengers |> head(n = 42), guinea_rice$Production)

autoplot(aus_airpassengers) +
  labs(title = "Australian air passengers") -> p1
autoplot(guinea_rice) +
  labs(title = "Guinea rice production") -> p2

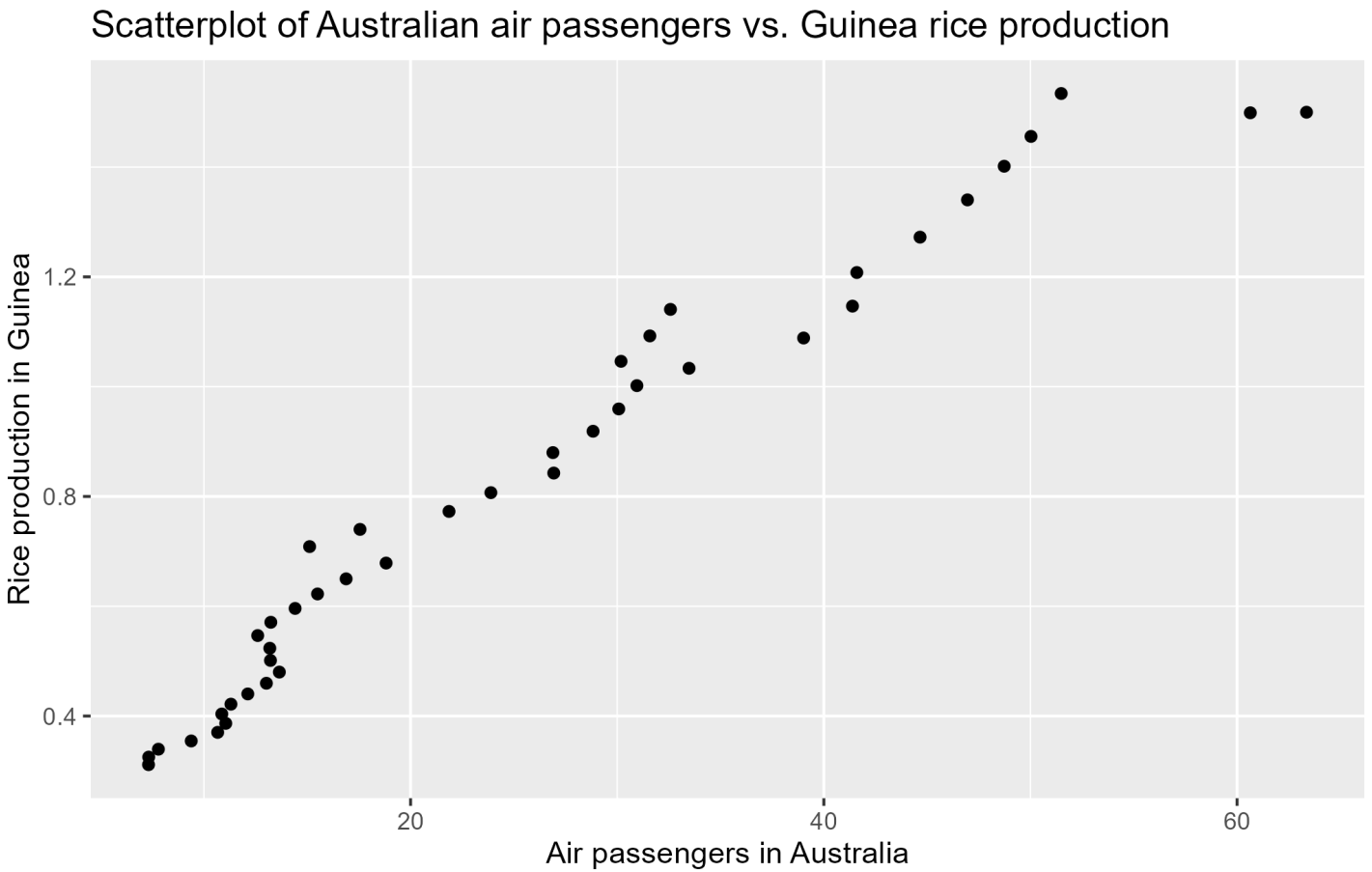
plot_grid(p1, p2, nrow = 2) #requires library cowplot
```


Australian air passengers
Passengers
60
40
20
1970
1980
1990
2000
2010
Year [1Y]


Guinea rice production
Production
1.2
0.8
0.4
1970
1980
1990
2000
2010
Year [1Y]

```r
#merge them in one tsibble
joined <- aus_airpassengers |>
  head(n = 42) |>
  left_join(guinea_rice, by = "Year")

joined |>
  ggplot(mapping = aes(x = Passengers, y = Production)) +
  geom_point() +
  labs(x = "Air passengers in Australia",
    y = "Rice production in Guinea",
    title = "Scatterplot of Australian air passengers vs Guinea rice production"
     )
```


Scatterplot of Australian air passengers vs. Guinea rice production
Rice production in Guinea
1.2
0.8
0.4
20
40
60
Air passengers in Australia

- Looking just at the plots, it looks like a linear regression model would be a great option.

```r
model <- joined |>
  model(model = TSLM(Passengers ~ Production))

model |> report()
```

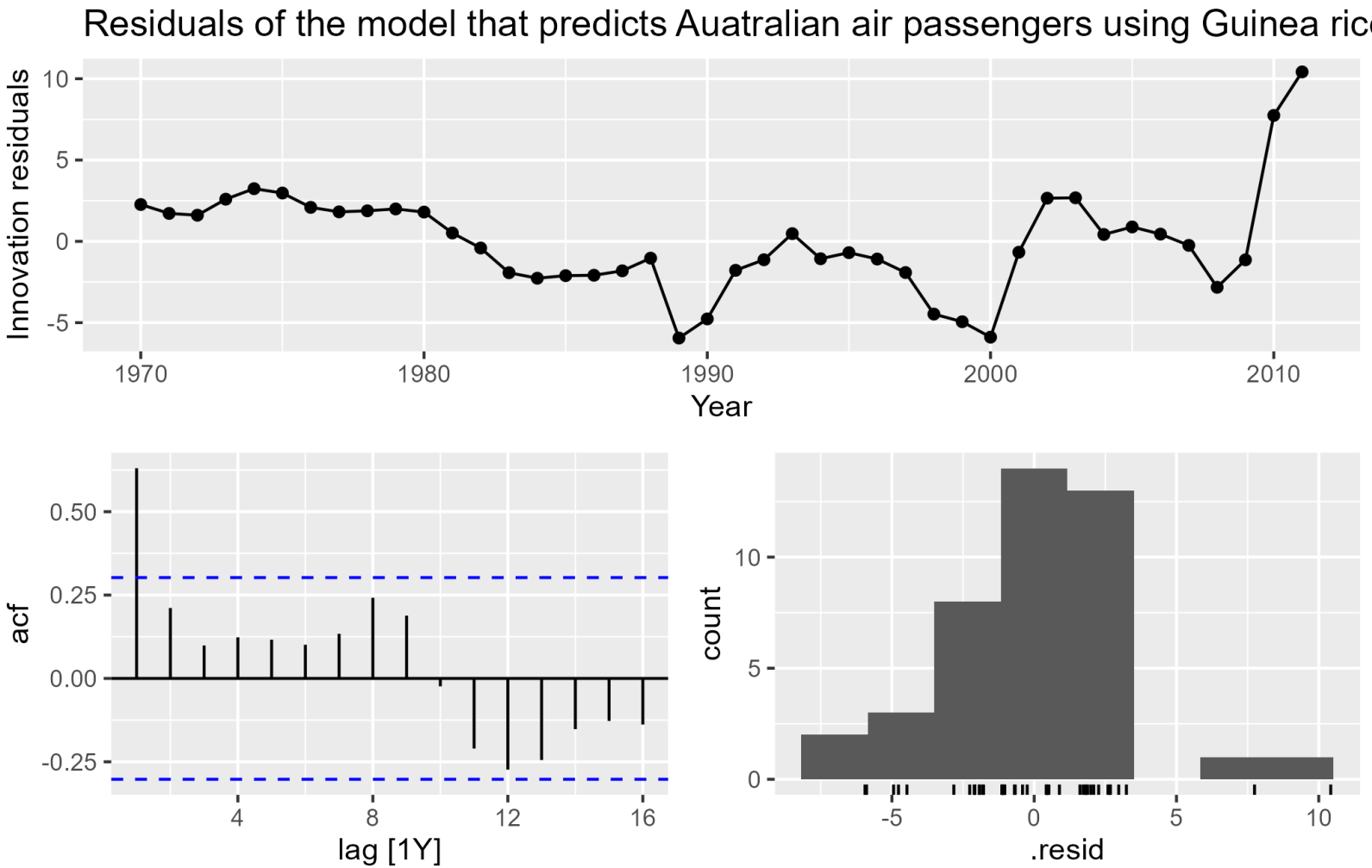
```
Series: Passengers
Model: TSLM

Residuals:
    Min      1Q  Median      3Q     Max
-5.9448 -1.8917 -0.3272  1.8620 10.4210

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   -7.493      1.203  -6.229 2.25e-07 ***
Production    40.288      1.337  30.135  < 2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 3.239 on 40 degrees of freedom
Multiple R-squared: 0.9578, Adjusted R-squared: 0.9568
F-statistic: 908.1 on 1 and 40 DF, p-value: < 2.22e-16
```

```
model |> gg_tsresiduals() +
  labs(title = "Residuals of the model that predicts Australian air passengers
      using Guinea rice production")
```


Residuals of the model that predicts Auatralian air passengers using Guinea rice
Innovation residuals
10
5
0
-5
1970
1980
1990
2000
2010
Year
acf
0.50
0.25
0.00
-0.25
4
8
12
16
lag [1Y]
count
10
5
0
-5
0
5
10
.resid

```
model |>
  augment() |>
  features(.innov, ljung_box, lag = 8) #lag should be minimum between 10 and T/5
```

```
.model lb_stat lb_pvalue
  <chr>    <dbl>     <dbl>
1 model     26.5  0.000867
```

- When we see a high R^2 value but also high residual autocorrelation, it is a sign of spurious regression.
- Even if for short periods spurious regression might provide good forecasts, obviously, on the long term they will not have good performance.

Useful predictors for time series

- In the previous example, we assumed that we have a data set made of several time series which can be used as predictors.
- Now we will look at what other predictors might be useful.

Trend as predictor I

- If the time series has a trend, we can create a feature which is simply the time of an observation and use it as predictor.

$$y_t = \beta_0 + \beta_1 * t + \epsilon_t$$

- In R this can be done by specifying *trend()* as predictor in the TSLM model.

```
air_model <- aus_airpassengers |>
  model(m = TSLM(Passengers ~ trend()))
air_model |>
  report()
```

Trend as predictor II

```
Series: Passengers
Model: TSLM

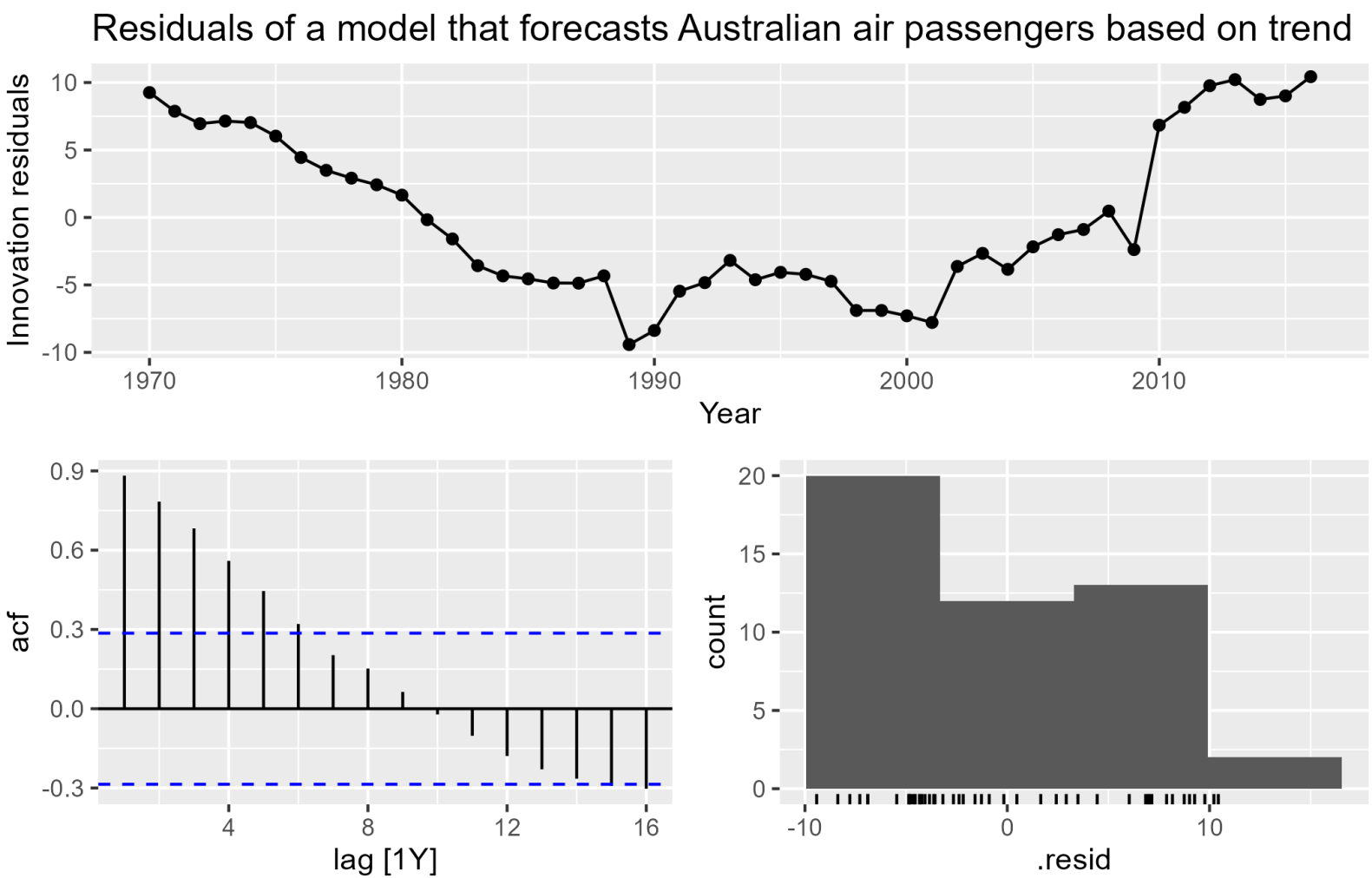
Residuals:
   Min     1Q Median     3Q    Max
-9.422 -4.582 -2.176  6.434 10.435

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -3.33603    1.78943  -1.864   0.0688 .
trend()      1.39360    0.06491  21.470   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 6.036 on 45 degrees of freedom
Multiple R-squared: 0.9111, Adjusted R-squared: 0.9091
F-statistic:   461 on 1 and 45 DF, p-value: < 2.22e-16
```

```
air_model |>
  gg_tsresiduals() +
  labs(title= "Residuals of a model that forecasts Australian air passengers
      based on trend")
```

Trend as predictor III



Dummy variables I

- What happens when we have categorical data in a data set and we want to use it as predictor? For example, in a sales-related time series an attribute specifying if the given day was a public holiday or not. This attribute would have the values *yes* or *no*.
- This situation can be handled by adding a dummy variable, which takes the values 0 (ex. for *no*) and 1 (for *yes*).

- Dummy variables can also be used to deal with outliers. Sometimes outliers are bad data (possibly registered incorrectly) but sometimes outliers are valid values that appear due to some special conditions (for example: tourist arrivals to a specific city might have outlier values if some special events - like the Olympics - are held in the city). In this case, instead of removing the values, it is easier to add a dummy variable to take value 1 for the outlier instance(s) and 0 for regular ones.

Dummy variables II

- If there are more than two possible values for the attribute, we need several dummy variables. More exactly, we need one less dummy variables than the number of possible values, since one of the values will be encoded with all zeros.
- For example, if we have daily data, we might want to consider the day of the week as a predictor. In this case, we need 6 dummy variables, and this is one example of how we can use them:
  - Monday - first dummy is 1, the others are zero
  - Tuesday - second dummy is 1, the others are zero
  - Wednesday - third dummy is 1, the others are zero
  - ...
  - Saturday - last dummy is 1, the others are zero
  - Sunday - all dummies are 0

Dummy variables III

- We need one less dummy, because the regression model that we will build also contains an intercept (the term β_0) which will measure the effect of the variable with all zeros.
- The interpretation of the coefficient that the regression will assign to each dummy is that it is a measure of the effect of that category *relative to the omitted category*.
- The coefficient that will be assigned to the first dummy in the previous example (the one which has a value of 1 for Monday) will show the relative influence of Mondays compared to Saturday.
- We could create dummies similarly for months in case of monthly data, or quarters in case of quarterly data. They are called *seasonal dummy variables*.

Dummy variables IV

- If you provide categorical features as predictors to the TSLM function, it will automatically create dummy variables for it. If you want to use seasonal dummy variables as predictors, you can use the *season* special as a predictor in the TSLM model.

Example I

- Let's consider the Australian beer production time series that we have already used and fit a linear regression model on the data from 1992 considering the trend and dummies for the quarters. This means that our model is going to be:

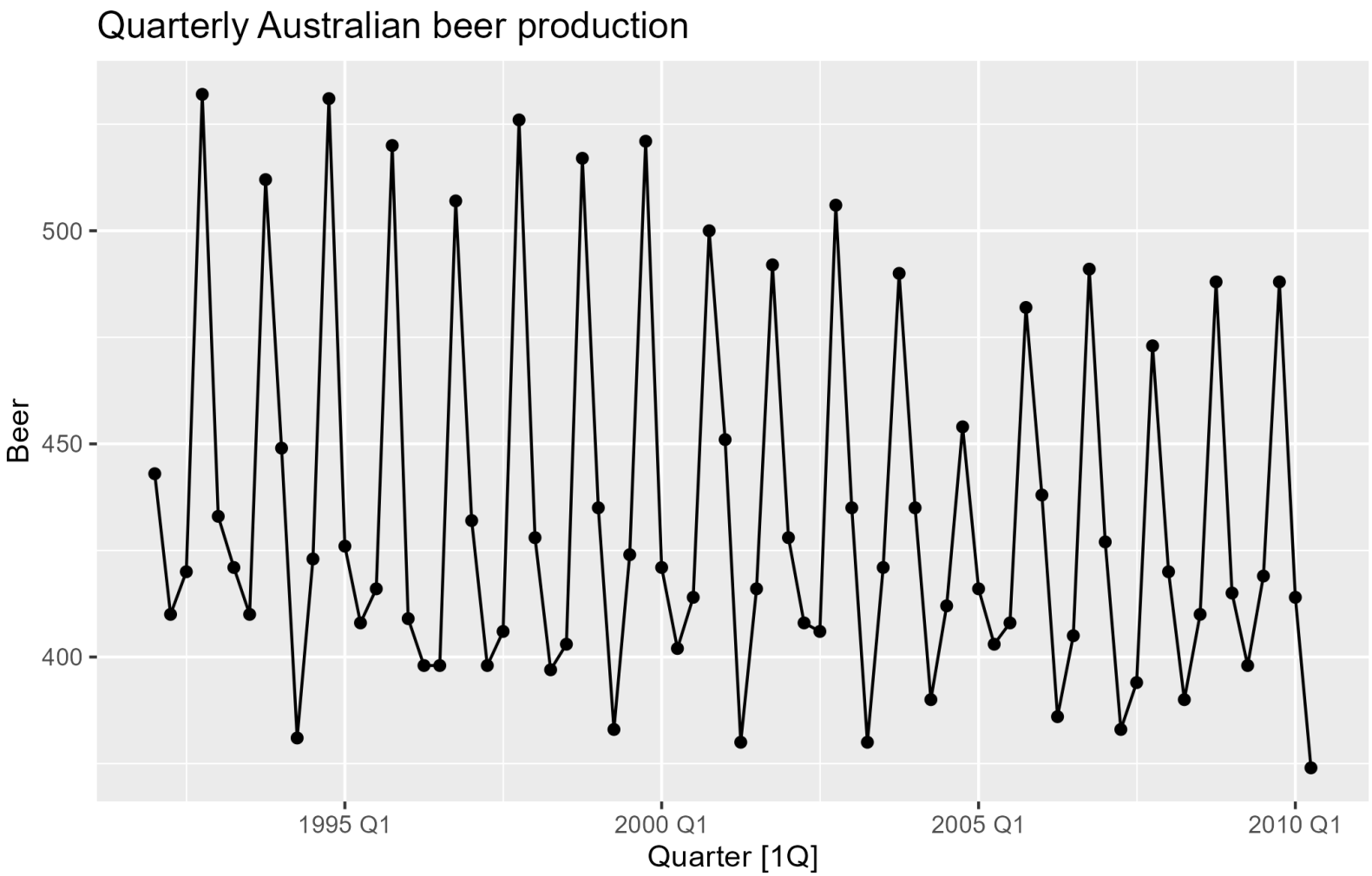
$$y_t = \beta_0 + \beta_1 * t + \beta_2 * d_{2,t} + \beta_3 * d_{3,t} + \beta_4 * d_{4,t} + \epsilon_t$$

- where $d_{i,t}$ is 1 if t is in quarter i. This means that we do not have a dummy for the first quarter, so the coefficients of the dummies will measure the effect of the specific quarter relative to the first one.

```
recent_production <- aus_production |>
  filter(year(Quarter) >= 1992)

recent_production |>
  autoplot(Beer) +
  labs(title = "Quarterly Australian beer production")
```

Example II


Quarterly Australian beer production
Beer
500
450
400
1995 Q1
2000 Q1
2005 Q1
2010 Q1
Quarter [1Q]

Example III

```
beer_fit <- recent_production |>
  model(TSLM(Beer ~ trend() + season()))
report(beer_fit)
```

```
Series: Beer
Model: TSLM

Residuals:
     Min       1Q   Median       3Q      Max
-42.9029  -7.5995  -0.4594   7.9908  21.7895

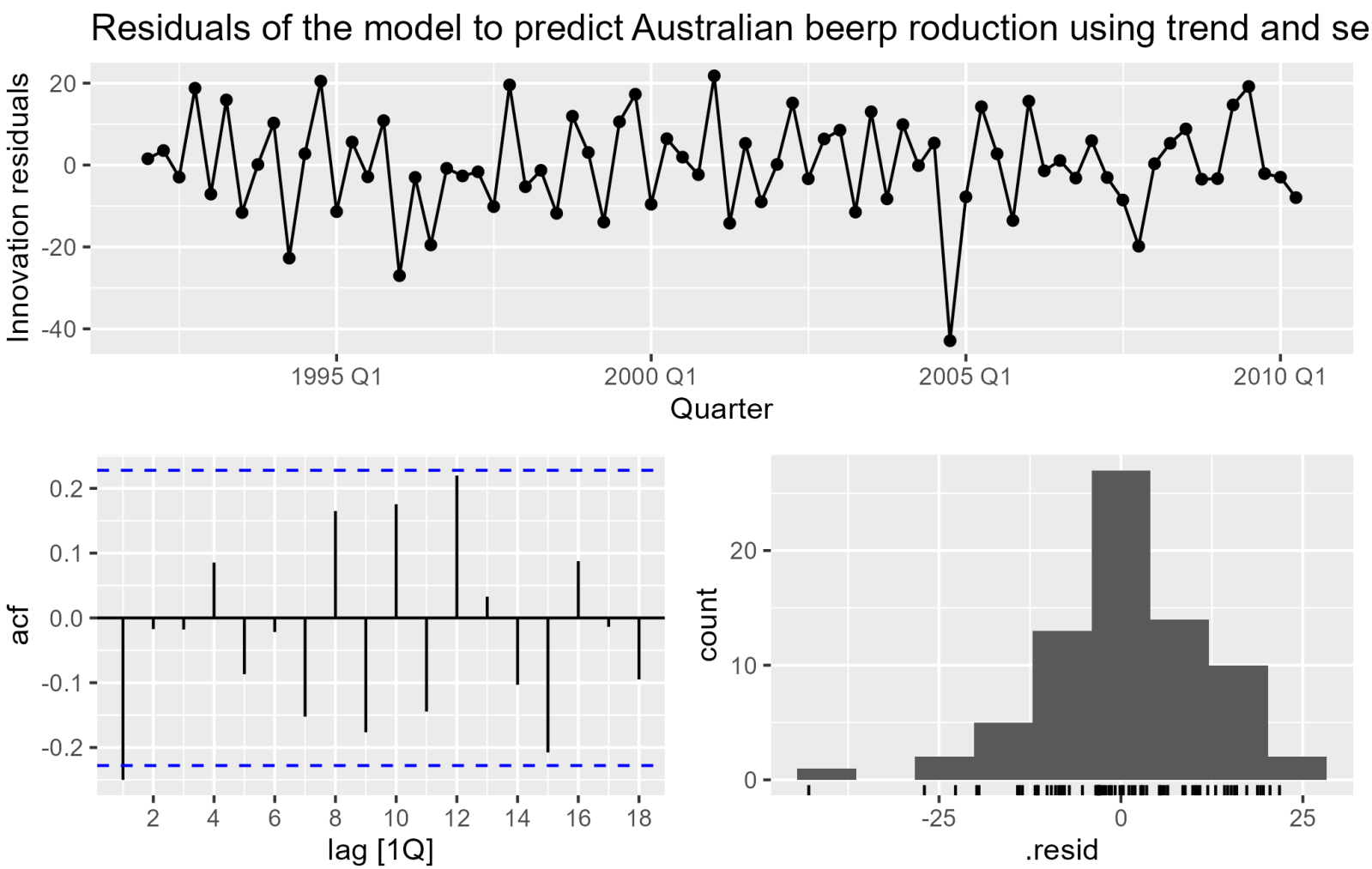
Coefficients:
               Estimate Std. Error t value Pr(>|t|)
(Intercept)   441.80044    3.73353 118.333  < 2e-16 ***
trend()        -0.34027    0.06657  -5.111 2.73e-06 ***
season()year2 -34.65973    3.96832  -8.734 9.10e-13 ***
season()year3 -17.82164    4.02249  -4.430 3.45e-05 ***
season()year4  72.79641    4.02305  18.095  < 2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 12.23 on 69 degrees of freedom
Multiple R-squared: 0.9243, Adjusted R-squared: 0.9199
F-statistic: 210.7 on 4 and 69 DF, p-value: < 2.22e-16
```

Example IV

```
beer_fit |>
  gg_tsresiduals() +
  labs(title = "Residuals of the model to predict Australian beer production
      using trend and seasonal dummies")
```

Example V


Residuals of the model to predict Australian beerp roduction using trend and se
Innovation residuals
20
0
-20
-40
1995 Q1
2000 Q1
2005 Q1
2010 Q1
Quarter
acf
0.2
0.1
0.0
-0.1
-0.2
2
4
6
8
10
12
14
16
18
lag [1Q]
count
20
10
0
-25
0
25
.resid

Example VI

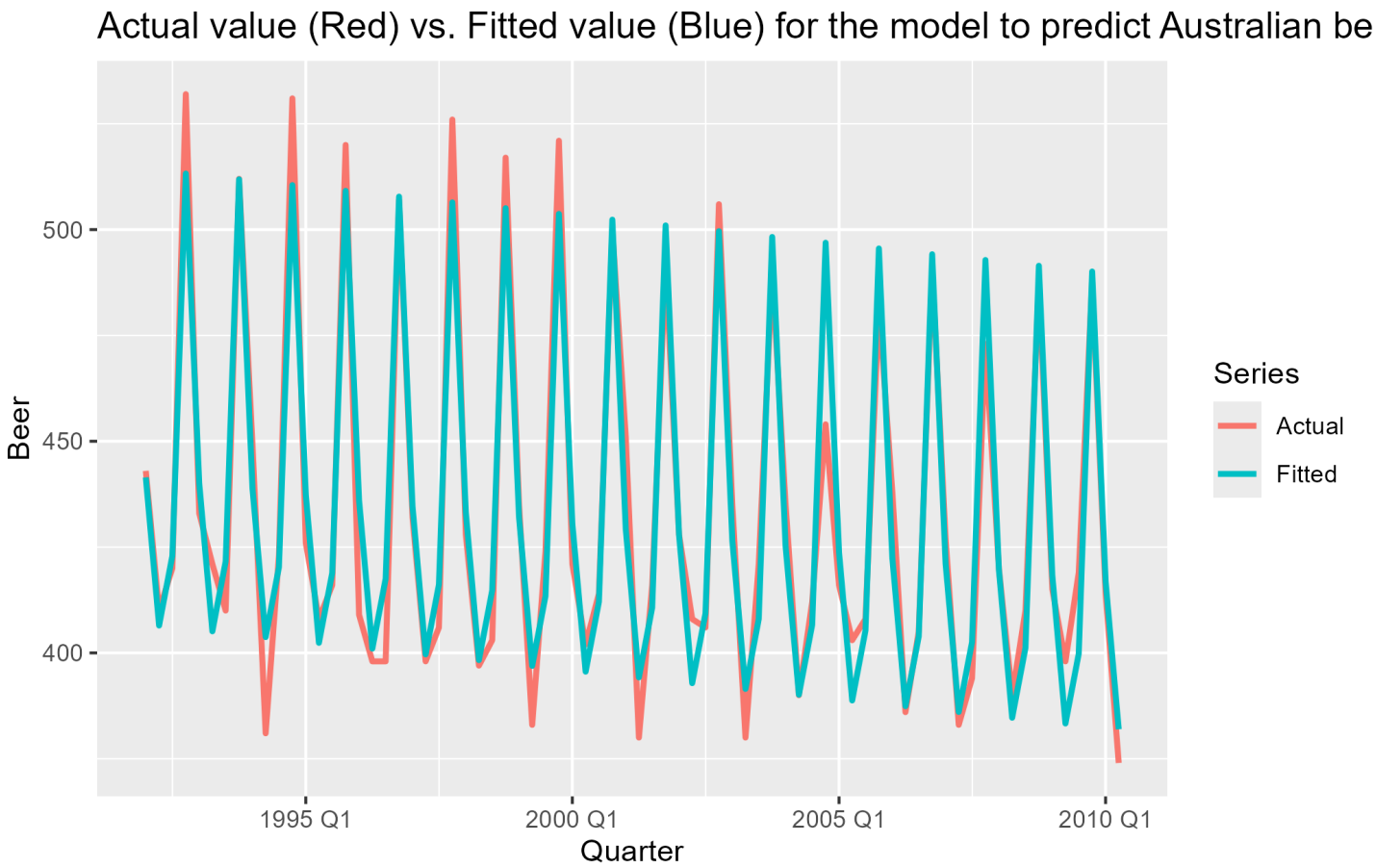
```
beer_fit |>
  augment() |>
  features(.innov, ljung_box, lag = 8) #white noise
```

```
 .model                           lb_stat lb_pvalue
  <chr>                             <dbl>     <dbl>
1 TSLM(Beer ~ trend() + season())    10.4     0.240
```

- Let's look at the fitted values, how they match the actual data.

```
beer_fit |> augment() |>
  ggplot(mapping = aes(x = Quarter)) +
  geom_line(mapping = aes(y = Beer, colour = "Actual"), linewidth = 1) +
  geom_line(mapping = aes(y = .fitted, colour = "Fitted"), linewidth = 1) +
  guides(colour = guide_legend(title = "Series")) +
   labs(title = "Actual value (Red) vs. Fitted value (Blue) for the model to
     predict Australian beer production using trend and seasonal dummies")
```

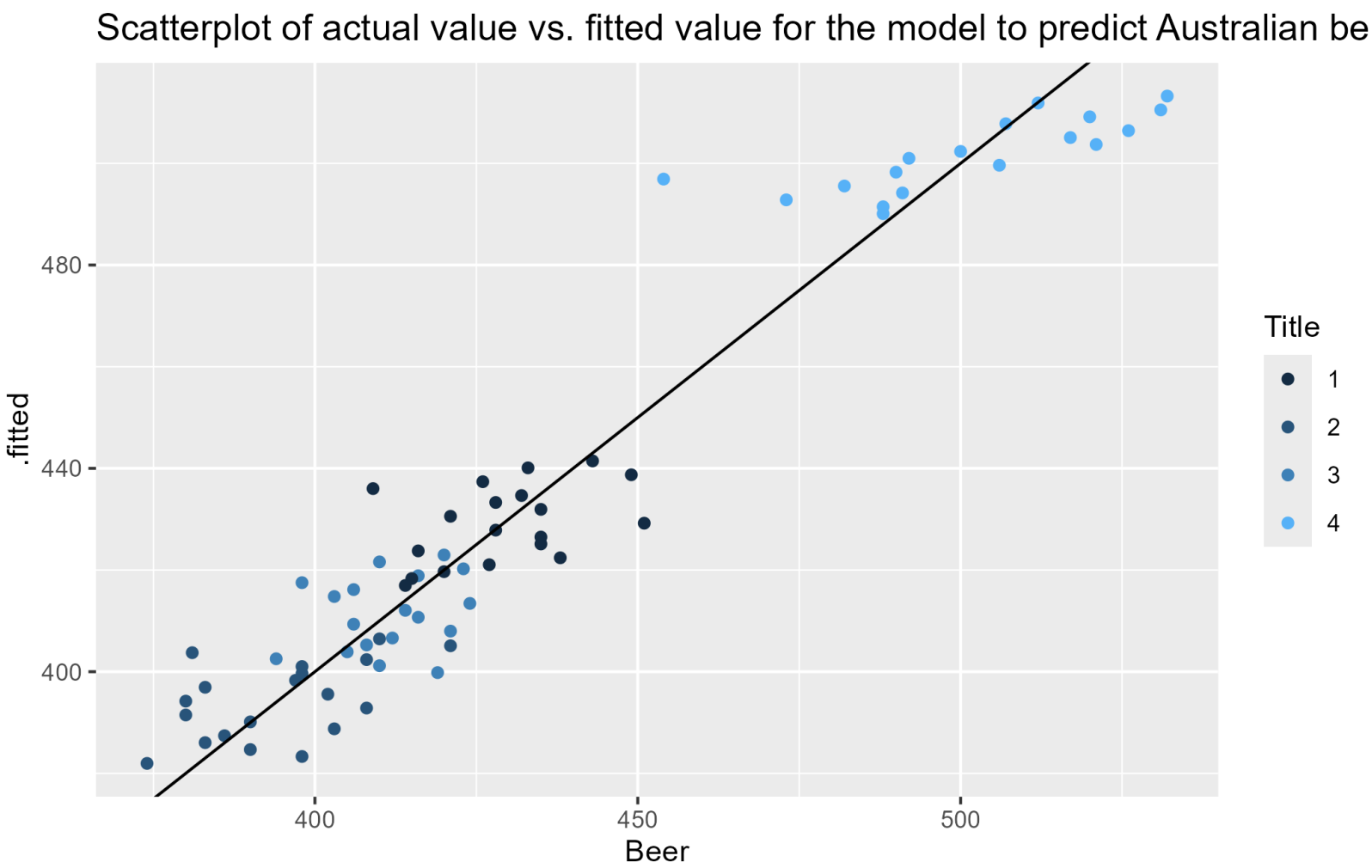
Example VII


Actual value (Red) vs. Fitted value (Blue) for the model to predict Australian be
Beer
500
450
400
1995 Q1
2000 Q1
2005 Q1
2010 Q1
Quarter
Series
Actual
Fitted

Example VIII

```
beer_fit |> augment() |>
    ggplot() +
    geom_point(mapping = aes(x = Beer, y = .fitted, colour = quarter(Quarter)))
     +
    geom_abline(intercept = 0, slope = 1) +
    guides (colour = guide_legend("Title")) +
    labs(title = "Scatterplot of actual value vs. fitted value for the model to
     predict Australian beer production using trend and seasonal dummies")
```

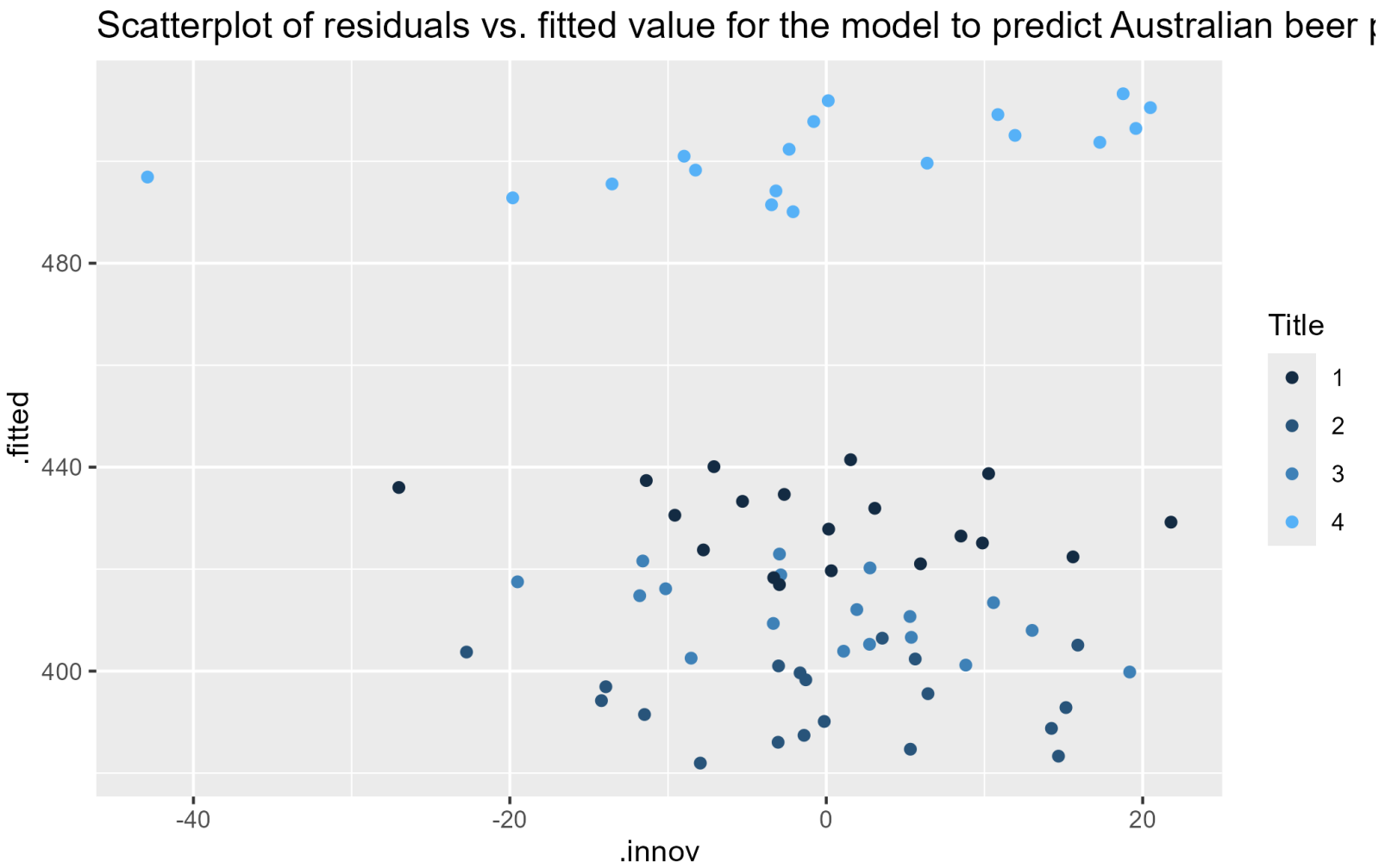
Example IX


Scatterplot of actual value vs. fitted value for the model to predict Australian be
.fitted
480
440
400
400
450
500
Beer
Title
1
2
3
4

Example X

```
beer_fit |> augment() |>
  ggplot() +
  geom_point(mapping = aes(x = .innov, y = .fitted, colour = quarter(Quarter)))
    +
  geom_abline(intercept = 0, slope = 1) +
  guides (colour = guide_legend("Title")) +
  labs(title = "Scatterplot of residuals vs. fitted value for the model to
    predict Australian beer production using trend and seasonal dummies")
```

Example XI


Scatterplot of residuals vs. fitted value for the model to predict Australian beer p
480
440
400
.fitted
-40
-20
0
20
.innov
Title
1
2
3
4

Example XII

- Pattern in the scatterplot of the fitted values vs. the actual data shows good fit.
- Lack of pattern in the scatterplot of the residuals vs. the fitted value also shows a good fit.

Other dummy variables I

- There are other situations when we can add dummy variables in our data set.
- **Intervention variables** - when something happened that influences the value of the forecast variable (very close to the idea of outliers). This can be of several types:
  - *spike* - the special effect lasts for one period only. The dummy takes 1 for this period and zero for the others.
  - *step* - an intervention which causes a level shift. The step variable is 0 before the shift and 1 after it.
  - change of slope in the trend
- **Trading days** - in case of monthly data, it might be worth adding a dummy with the number of trading days for that month. Alternatively, instead of just the number of trading days, we could have seven dummies denoting the number of Mondays, Tuesdays, etc. in that month.

Other dummy variables II

- **Lags** - for factors which might make their effect later (for example, advertising expenditure) it might be worth including the lagged values of that factor.

- **Easter** - Easter is a special holiday because it is not always on the same day (not even same month). We can use a dummy for the day(s) of Easter, or, in case of monthly data, we can add two dummies to show if Easter is in March or April.
  - **Obs.** In this case, we need two dummies, since technically we have three cases: Easter in March, Easter in April, Easter in both (in which case the values in the dummy are proportionally split).

Selecting predictors I

- Sometimes, we have a lot of possible predictors (especially when we consider dummies) and we need a strategy to select the ones to be used in the regression model.
- A **bad** strategy is to create scatterplots of each possible predictor variable and the forecast variable and drop everything where no relationship can be seen. Sometimes the relationship cannot be identified from the plot.
- Another **bad** strategy is to build a model including all predictors and look at their p-values (in the output of the *report* function) and discard predictors with small p-values.
- A **good** approach is to use the the so-called *measures of predictive accuracy*, which can be seen using the *glance* function on a model.

Selecting predictors II

```
us_change |>
  model( all = TSLM(Consumption ~ Income + Savings + Unemployment + Production))
     |>
  glance() -> measures
measures
measures <- measures |>
  select(adj_r_squared, CV, AIC, AICc, BIC)
measures
```

```
  adj_r_squared    CV   AIC  AICc   BIC
  <dbl>           <dbl> <dbl> <dbl> <dbl>
1 0.763           0.104 -457. -456. -437.
```

- Out of the selected 5 measures, *adjusted* R^2 should be maximized while the other 4 should be minimized.

Adjusted R^2

- The R^2 reported by the *report* function for a model was computed on the same data which was used for building the model. It measures how well the model fits the historical data, but it does not tell us how well the model will forecast.
- Also, it was observed that by adding predictors to the model the value of R^2 will increase, even if the predictor is useless.
- A measure which overcomes these problems is the *adjusted* R^2, denoted by $\bar{R}^2$.

$$\bar{R}^2 = 1 - (1 - R^2) * \frac{T - 1}{T - k - 1}$$

- where T is the number of observations and k is the number of predictors in the model.

CV

- CV - comes from cross-validation.
- We have discussed in the previous lecture how to do cross-validation for time series, however, for regression it is possible to do classical *leave-one-out* cross validation, which is done in the following way:
  - Remove observation t from the data set and build a regression model on the rest of the observations. Predict the value of the removed observation and compute the corresponding error e_t.
  - Repeat for all observations, getting an error for each.
  - Compute the MSE of the errors. This is the value of CV.

Akaike's Information Criterion (AIC)

- AIC is defiend in the following way:

$$AIC = T * log(\frac{SSE}{T}) + 2 * (k + 2)$$

- where T is the number of observations, k is the number of predictors and SSE is the sum of the squared errors, computed in the following way:

$$SSE = \sum_{t=1}^{T} e_t^2$$

- For small values of T the AIC tends to select models with too many predictors, so a corrected version of AIC was proposed:

$$AIC_c = AIC + \frac{2(k+2)(k+3)}{T-k-3}$$

Schwarz's Bayesian Information Criterion (BIC)

- A related measure is BIC, computed in the following way:

$$BIC = T * log(\frac{SSE}{T}) + (k + 2) * log(T)$$

- BIC penalizes models with more predictors more than AIC does, so BIC will select either the same method as AIC or one with fewer terms.

Which one to use?

- While $\bar{R}^2$ is frequently used, it tends to find models with many predictors.
- AICc, AIC and CV are recommended to be used, since they have forecasting as their objective. If T is large enough, they will all lead to the same model.

Selecting the predictors

- If possible, all possible combinations of predictor variables should be fitted and then the best one can be selected.
- However, if the number of possible predictors is large, this might not be an option. In this case a strategy is needed to limit the number of combinations to experiment with.
- There are three approaches:
  - *backwards stepwise regression* - build a model on all predictors. Eliminate one predictor at a time. Keep the new model with the best performance (if it is better than the one with all predictors). Repeat until no more improvement happens.
  - *forward stepwise regression* - start with a model having only an intercept. Add one-by-one all predictors, keep the best model. Repeat until no further improvement is possible.
  - **hybrid** procedure - start with some predictors already then either include new ones or remove existing ones.
- The above presented stepwise approaches do not guarantee that the best model is found, but they almost always find a good model.

How to forecast? I

- We saw many things to do/check considering the fitted values. But how to forecast *future* values?
- There are two types of forecasts: *ex-ante* and *ex-post* forecasts.
- To understand the difference between them, let's look at the example with the Consumption again and assume we have the model where Consumption is computed using the other four attributes.

How to forecast? II

- In **ex-ante** forecast, we build the model using only information which is available in advance. In our case it means to build a model from all data available (up until 1992 Q2). This means that when we want to forecast Consumption, we first need to forecast the other attributes. This can be done with simple or more sophisticated methods.
- In **ex-post** forecasting, later information about the predictors is considered. This is not genuine forecasting, but might be useful as analysis.

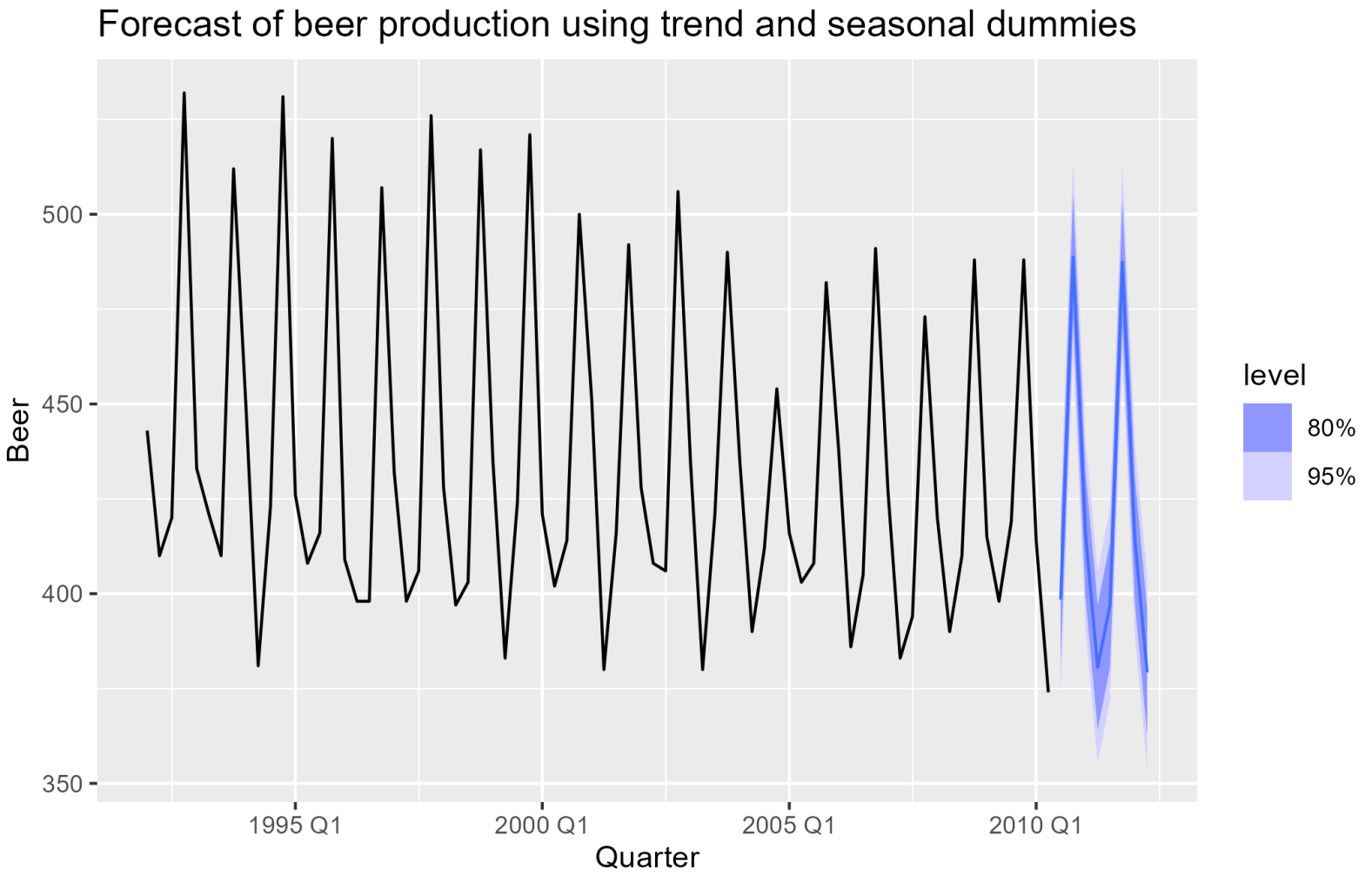
Example I

- Let's consider the beer production again.

```
recent_production <- aus_production |>
  filter(year (Quarter) >= 1992)

beer_fit <- recent_production |>
  model(TSLM(Beer ~ trend() + season()))
beer_forecast <- forecast(beer_fit)
beer_forecast |>
  autoplot(recent_production) +
  labs(title = "Forecast of beer production using trend and seasonal dummies")
```

Example II



- The *trend* and the *season* are dummy variables and they are known in advance, so we have genuine forecasts.

Scenario-based forecasting I

- In this version, the predictor variables are estimated by simply considering different scenarios for them.
- For example, how does a future increase or decrease in Income and Savings influence the value of Consumption?

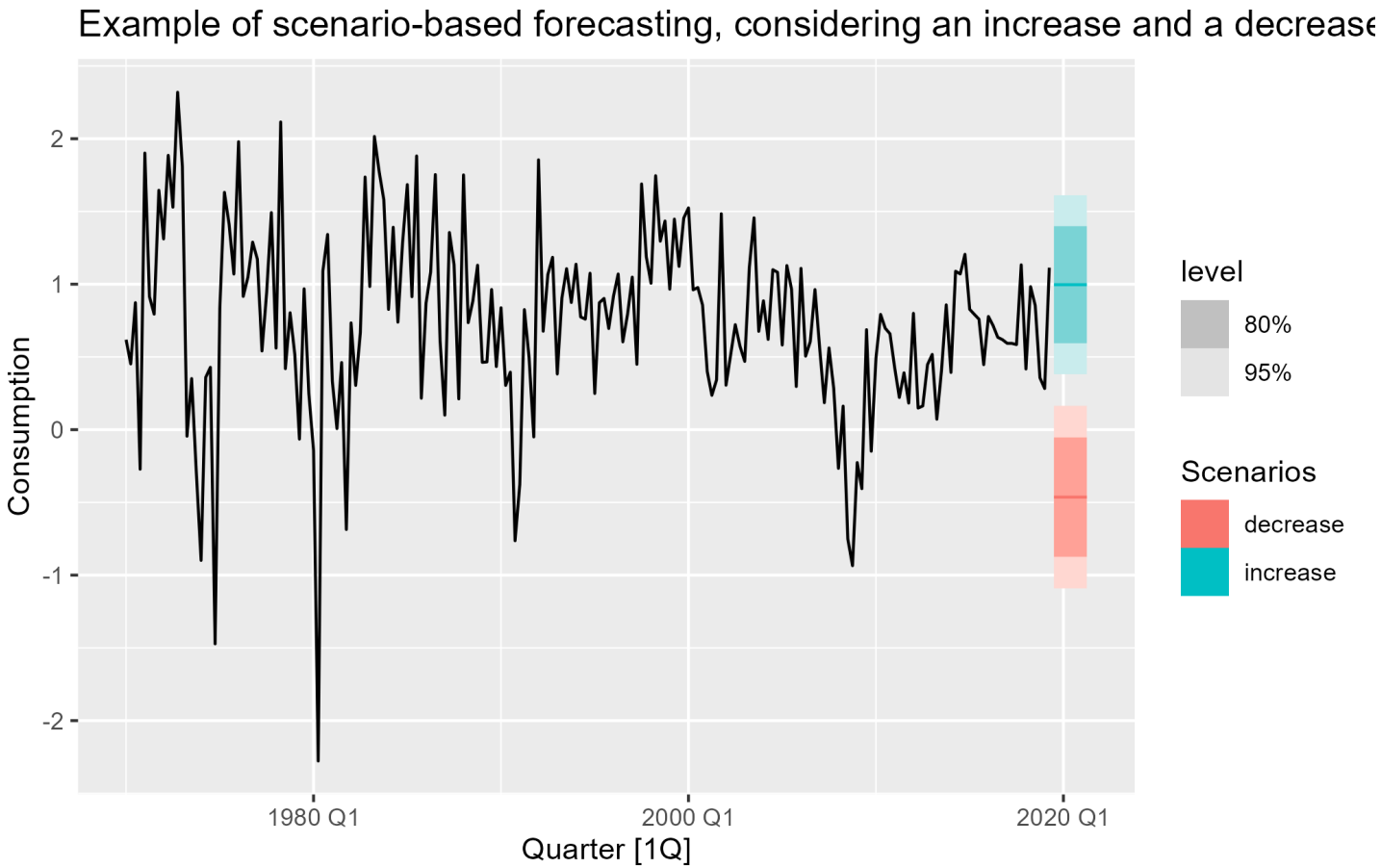
```
consumptionFit <- us_change |>
  model(all = TSLM(Consumption ~ Income + Savings + Unemployment))

future_scenarios <- scenarios(
  increase = new_data(us_change, 4) |>
    mutate(Income = 1, Savings = 0.5, Unemployment = 0),
  decrease = new_data(us_change, 4) |>
    mutate(Income=-1, Savings = -0.5, Unemployment = 0),
  names_to = "Scenarios"
  )

consumptionForecast <- forecast(consumptionFit, new_data = future_scenarios)

us_change |>
  autoplot(Consumption) +
  autolayer(consumptionForecast)  +
  labs(title = "Example of scenario-based forecasting, considering an increase
     and a decrease scenario")
```

Scenario-based forecasting II



- Why are the forecasts flat (why are they a straight line)?

Non-linear regression I

- Sometimes a linear model is not enough and we need a non-linear model.
- The simplest solution is to transform the predictor and/or the forecast variable (for example, using the log transformation) and then use linear regression

$$log\ y_t = \beta_0 + \beta_1 * log\ x_t + \epsilon_t$$

- The above example is called **log-log** functional form, because we took the logarithm from both terms.
- There is **log-linear** form, when only the forecast variable is transformed and **linear-log** form when only the predictor variable is transformed.
- One possible issue with the logarithm is that the values have to be greater than zero. If x contains 0, we can use $log(x+1)$ as transformation, with the nice property that the transformed value will be 0 when x is 0.

Non-linear regression II

- When a simple logarithmic transformation is not enough, we can consider a more general non-linear function f:

$$y_t = f(x_t) + \epsilon_t$$

- One of the simplest cases is when we consider that f is *piecewise linear*, meaning that it is linear in segments, but there are points when the slope of the line changes. These points are called **knots**.
- Assuming that there is only one *knot* at point c, we can deal with it, in the following way:
  - We introduce $x_1 = x$ (just renaming it, to have indexes)
  - We introduce $x_2 = (x - c)_+$, where $(x - c)_+$ is $x - c$ if the value is positive and it is zero otherwise.
- If we have more knots, we can add more attributes in the similar manner, however, finding the knot points is not an easy thing.

Forecasting with a non-linear trend

- We have seen that we can use $x_{1,t} = t$ as a predictor variable to account for trend.
- If we think that the trend is not linear, one thing that we could do is to use quadratic or higher order trends, but this is not recommended.
- It is better to use piecewise linear models which bend at some point.
- If we think that the trend bends at time τ, we can do the following:
  - Use $x_{1,t} = t$
  - Introduce $x_{2,t} = (t - \tau)_+$, which is $(t - \tau)$ if the value is positive and 0 otherwise.

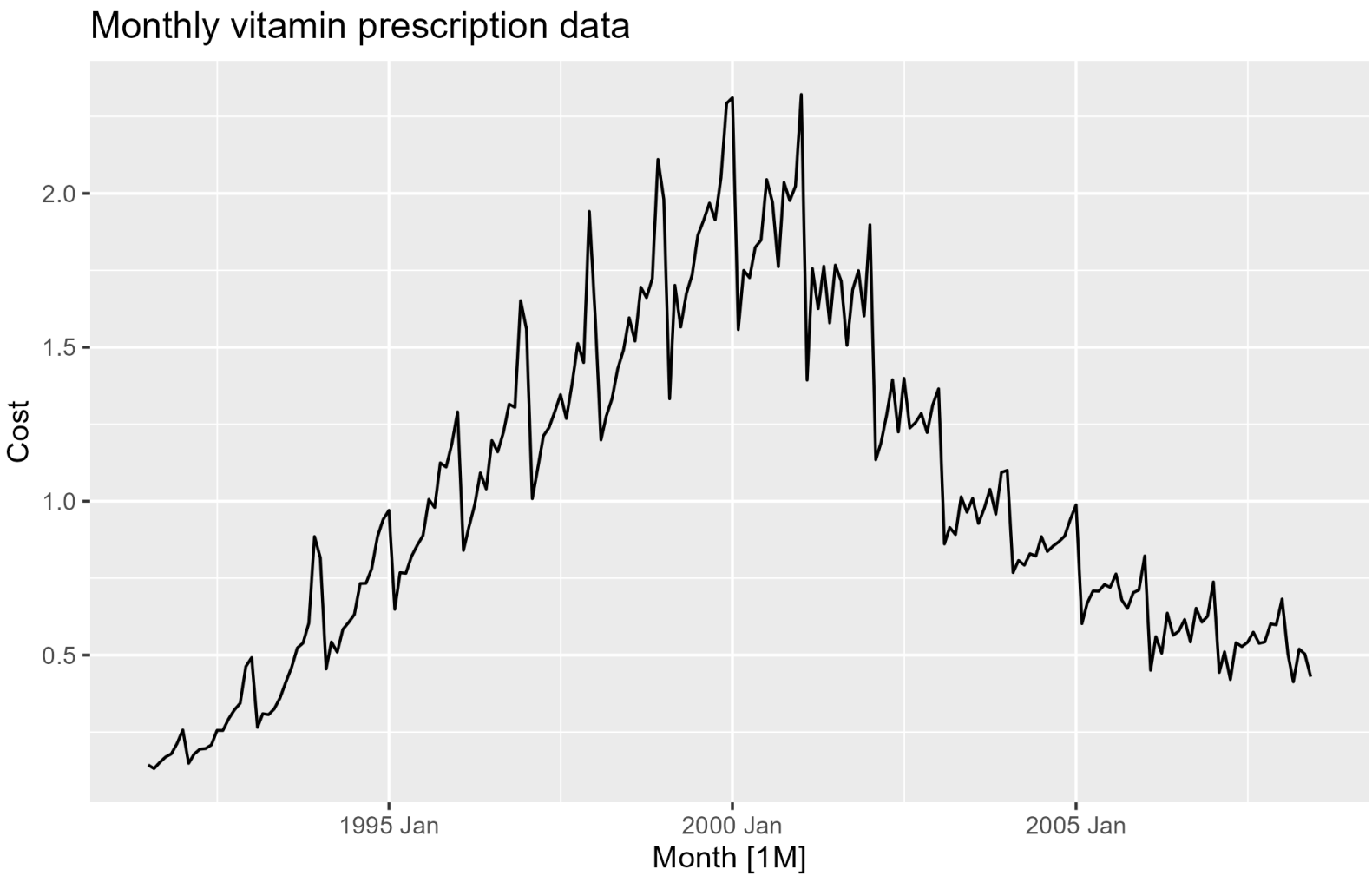
Example 1 I

- Remember the vitamin prescription data from Australia? It had one increasing trend and then at a given point the trend became decreasing.

```
vitamins <- PBS |>
  filter(ATC2 == "A11") |>
  select(Month, Concession, Type, Cost) |>
  summarize(TotalCost = sum(Cost)) |>
  mutate(Cost = TotalCost / 1000000)

autoplot(vitamins, Cost) +
  labs(title = "Monthly vitamin prescription data")
```

Example 1 II


Monthly vitamin prescription data
Cost
2.0
1.5
1.0
0.5
1995 Jan
2000 Jan
2005 Jan
Month [1M]

Example 1 III

```
vitamins_model1 <- vitamins |>
  model(TSLM(Cost ~ trend()))
vitamins_model1 |> report()
```

```
Series: Cost
Model: TSLM

Residuals:
     Min       1Q   Median       3Q      Max
-0.79551 -0.46137 -0.09127  0.36100  1.29657

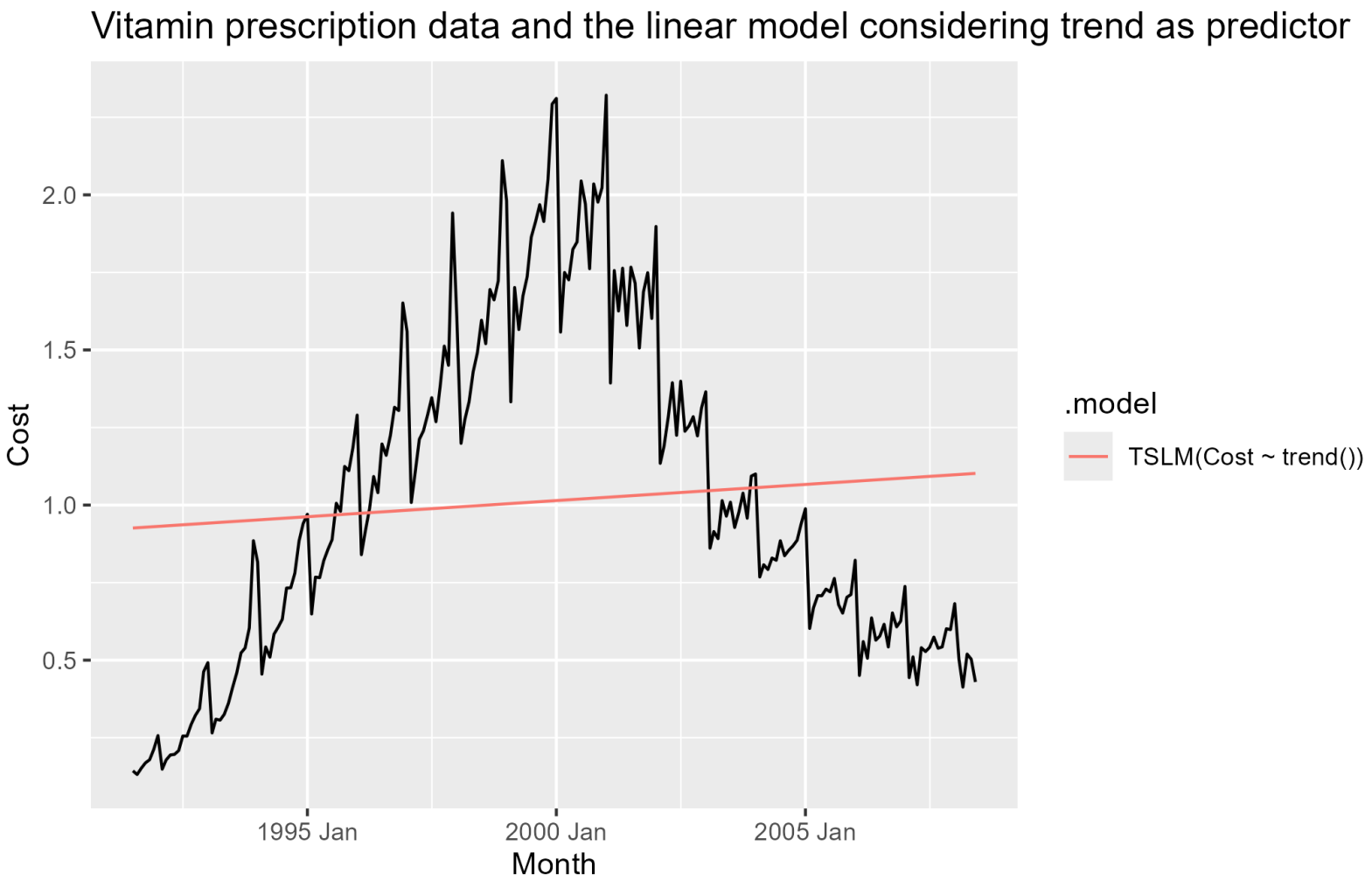
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 0.9251069  0.0755289  12.248   <2e-16 ***
trend()     0.0008671  0.0006389   1.357    0.176
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.5374 on 202 degrees of freedom
Multiple R-squared: 0.009035, Adjusted R-squared: 0.00413
F-statistic: 1.842 on 1 and 202 DF, p-value: 0.17626
```

Example 1 IV

```
vitamins |> ggplot(mapping = aes(x = Month, y = Cost)) +
  geom_line() +
  geom_line(data = fitted(vitamins_model1), aes(y = .fitted, colour = .model)) +
  labs(title = "Vitamin prescription data and the linear model considering trend
       as predictor")
```

Example 1 V


Vitamin prescription data and the linear model considering trend as predictor
Cost
2.0
1.5
1.0
0.5
1995 Jan
2000 Jan
2005 Jan
Month
.model
TSLM(Cost ~ trend())

Example 1 VI

```
vitamins_model2 <- vitamins |>
  model(TSLM(Cost ~ trend(knots = yearmonth("2000 Jan"))))
vitamins_model2 |> report()
```

```
Series: Cost
Model: TSLM

Residuals:
     Min       1Q   Median       3Q      Max
-0.42113 -0.10839 -0.01503  0.09709  0.62865

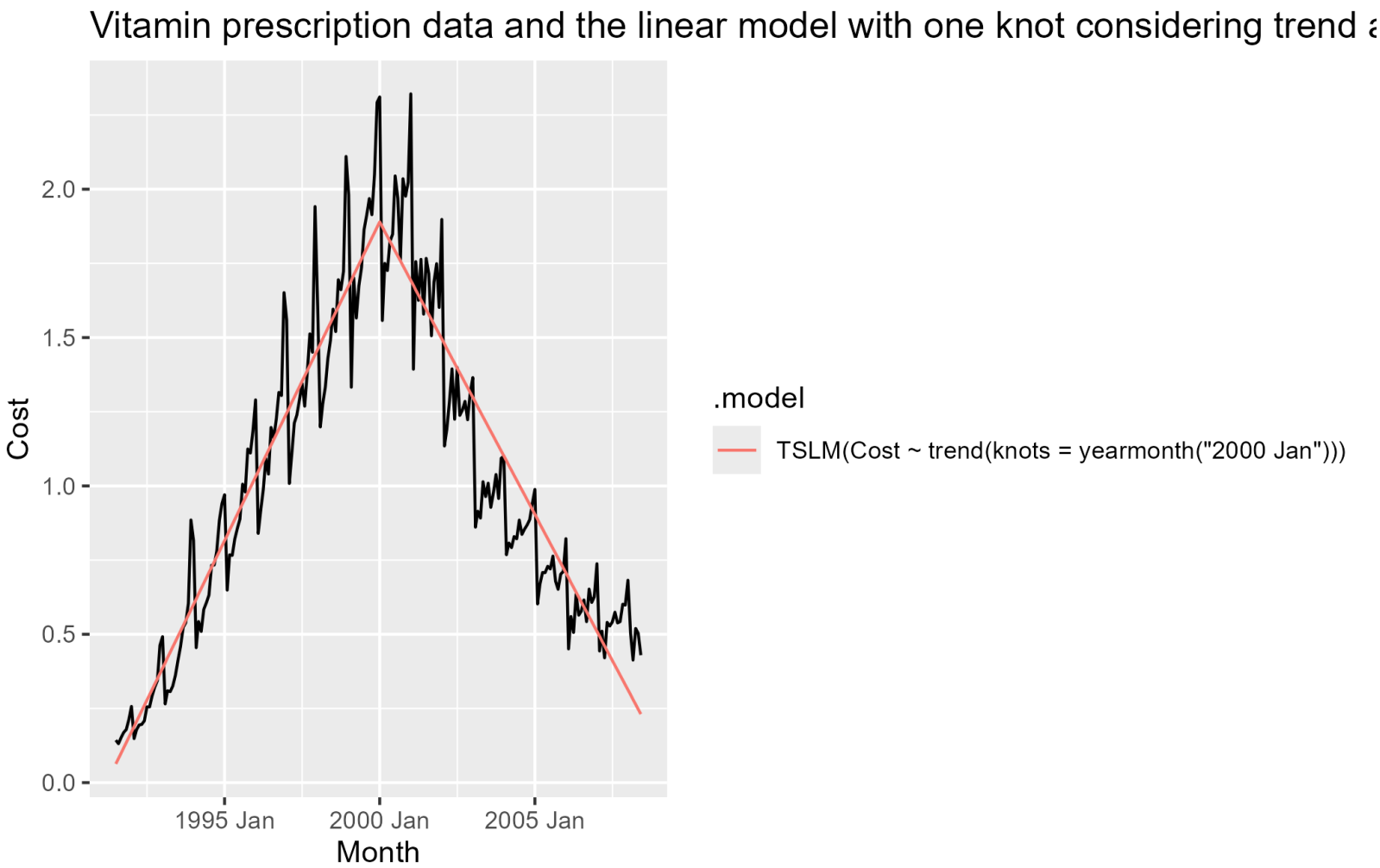
Coefficients:
                                                Estimate Std. Error t value Pr(>|t|)
(Intercept)                                    0.0454080  0.0327238   1.388    0.167
trend(knots = yearmonth("2000 Jan"))trend      0.0179070  0.0004649  38.515   <2e-16 ***
trend(knots = yearmonth("2000 Jan"))trend_103 -0.0343322  0.0008366 -41.037   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.1759 on 201 degrees of freedom
Multiple R-squared: 0.8943, Adjusted R-squared: 0.8933
F-statistic: 850.6 on 2 and 201 DF, p-value: < 2.22e-16
```

Example 1 VII

```
vitamins |> ggplot(mapping = aes(x = Month, y = Cost)) +
  geom_line() +
  geom_line(data = fitted(vitamins_model2), aes(y = .fitted, colour = .model)) +
  labs(title = "Vitamin prescription data and the linear model with one knot
      considering trend as predictor")
```

Example 1 VIII



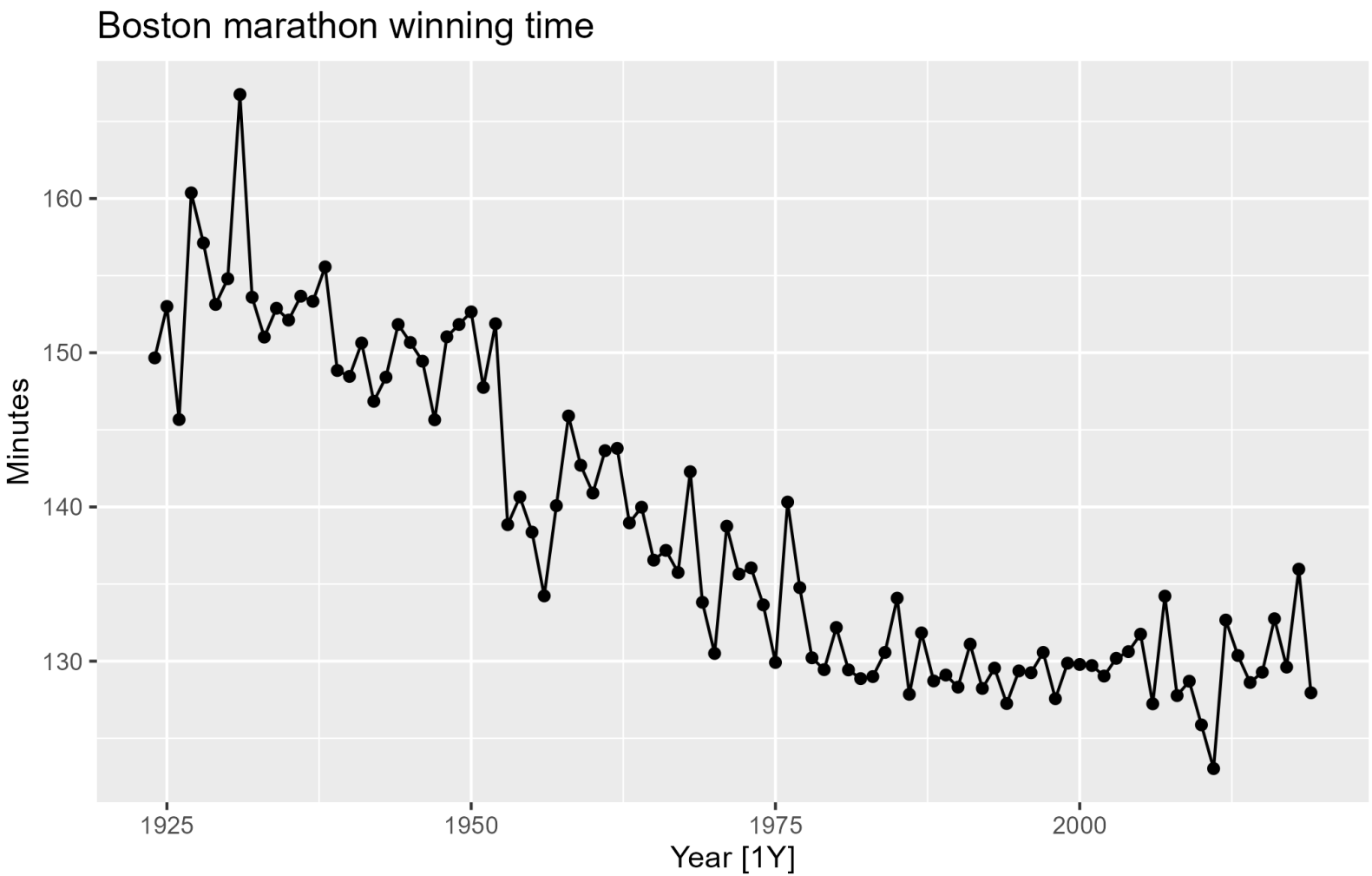
Example2 I

- Let's look at the winning times for men at the Boston marathon after 1924.

```
boston_marathon |>
  filter(Year >= 1924) |>
  filter(Event == "Men's open division") |>
  mutate(Minutes = as.numeric(Time)/60) ->boston_men

boston_men
boston_men |> autoplot(Minutes) +
  labs(title = "Boston marathon winning time")
```

Example2 II


Boston marathon winning time
Minutes
160
150
140
130
1925
1950
1975
2000
Year [1Y]

Example2 III

```
boston_model <- boston_men |>
  model(linear = TSLM(Minutes ~ trend()),
        logarithmic = TSLM(log(Minutes) ~ trend()),
        piecewise = TSLM(Minutes ~ trend(knots = c(1950, 1980)))
  )
boston_model |>
  accuracy() |>
  arrange(MAE)
```

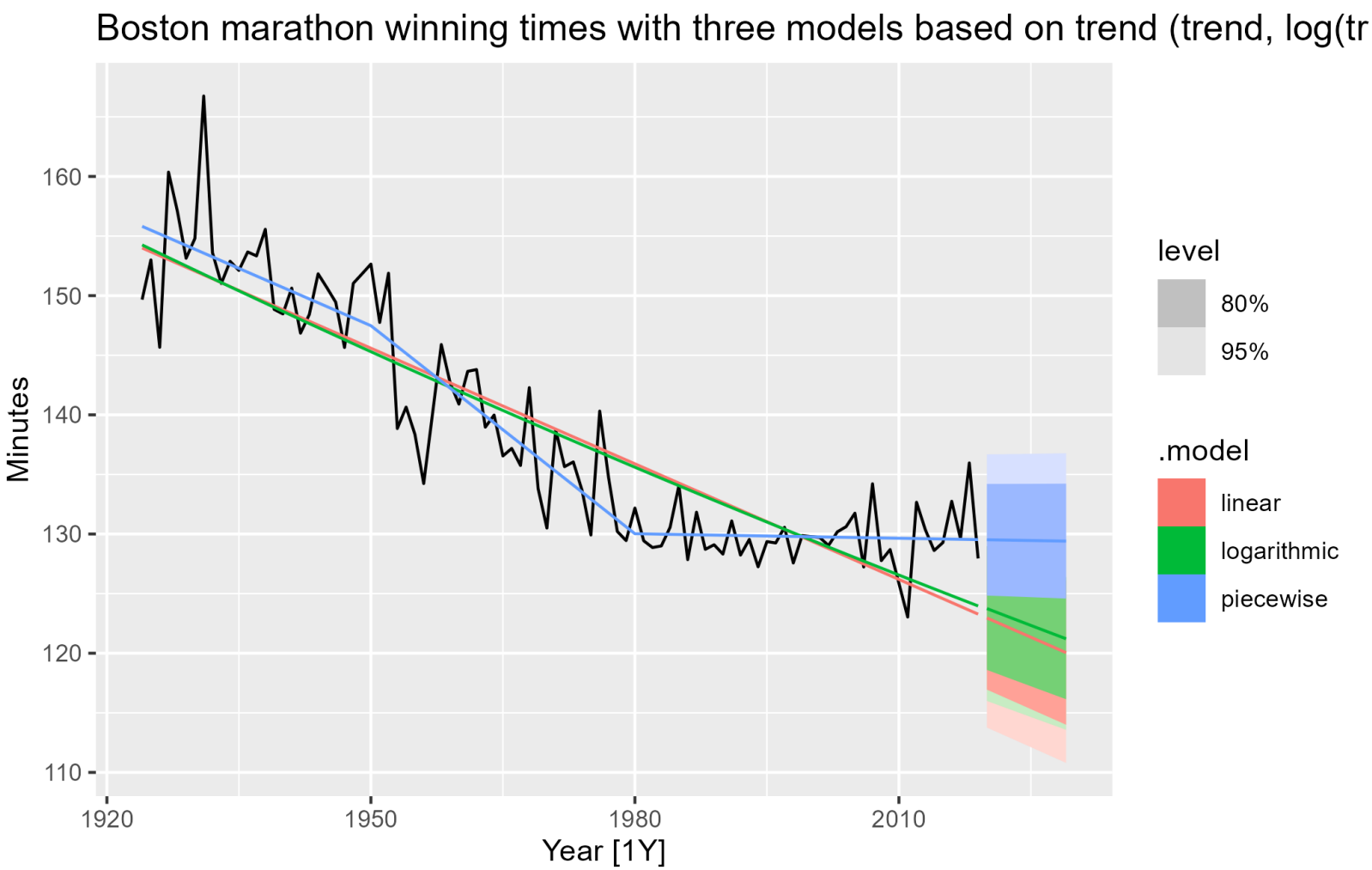
```
Event                .model    .type          ME  RMSE   MAE     MPE  MAPE  MASE RMSSE   ACF1
  <fct>                <chr>     <chr>       <dbl> <dbl> <dbl>   <dbl> <dbl> <dbl> <dbl>  <dbl>
1 Men's open division piecewise Training -2.66e-15  3.42  2.48 -0.0573  1.76 0.723 0.743 0.0901
2 Men's open division loga      Training  7.38e- 2  4.42  3.51 -0.0501  2.53 1.02  0.959 0.453
3 Men's open division linear    Training -2.96e-16  4.55  3.62 -0.100   2.62 1.05  0.987 0.483
```

Example2 IV

```
boston_forecast <- boston_model |> forecast(h = 10)
boston_forecast

boston_men |>
  autoplot(Minutes) +
  autolayer(boston_forecast) +
  geom_line(data = fitted(boston_model), aes(y = .fitted, colour = .model)) +
  labs(title = "Boston marathon winning times with three models based on trend (
      trend, log(trend), trend with 2 knots) and forecasts for 10 years")
```

Example2 V


Boston marathon winning times with three models based on trend (trend, log(tr
160
150
140
130
120
110
Minutes
1920
1950
1980
2010
Year [1Y]
level
80%
95%
.model
linear
logarithmic
piecewise